



Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Programming in Modern C++

Module M56: C++11 and beyond: Resource Management by Smart Pointers: Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Learnt how Rvalue Reference works as a Universal Reference under template type deduction
- Understood the problem of forwarding of parameters and its solution using Universal Reference and `std::forward`
- Understood how Move works as an optimization of Copy
- Understood λ expressions (unnamed function objects) in C++ with
 - Closure Objects
 - Parameters
 - Capture
- Learnt different techniques without or with `std::function` to write and use non-recursive and recursive λ expressions in C++11 / C++14
- Introducing several class features in C++11 with examples
- Explained how these features enhance OOP, generic programming, readability, type-safety, and performance in C++11
- Introduced several features in C++11 for non-class types and templates with examples
- Familiarizes with important non-class types like enum class and fixed width integer
- Familiarized with important templates like variadic templates



Module Objectives

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Revisit Raw Pointers for resource management
- Introduce Smart pointers with typical interface and use
- Introduce the policies for smart pointer



Module Outline

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

1 Weekly Recap

2 Raw Pointers

- Operations
- Ownership Issue
- Pointers vs. Reference

3 Smart Pointers

- Policies
 - Storage Policy
 - Ownership Policy

4 Module Summary



Raw Pointers

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Raw Pointers

Sources:

- [How to use C++ raw pointers properly?](#) Soroush Khajepor, 2020



Motivation: Resource Management

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Imbibe a culture to write **good** C++ code
 - *Correct*: Achieves the functionality
 - *Bug free*: Free of programming errors
 - *Maintainable*: Easy to develop and support
 - *High performance*: Fast, Low on memory
- Dynamic creation & destruction of objects is a *strength* and a *bugbear* of C / C++
- It needs manual *resource management* by the programmer. She / he has to control:
 - the allocation of memory for the object,
 - handle the object's initialisation and,
 - ensure that object was safely cleaned-up after use and its memory returned to heap
- This leads to C / C++ being an *unsafe, memory-leaking* language
- **Resource Management** frees the client from having to worry about the lifetime of the managed object, eliminating memory leaks and other problems in C++ code
- A resource could be any object that required dynamic creation/deletion – *memory, files, sockets, mutexes*, etc.
- Effective *Resource Management* is needed so that *dynamically managed objects can be managed as automatic object*



Raw Pointers: Operations

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Raw Pointers: Operations



Raw Pointer Operations

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Dynamic Allocation (result of) or `operator&`
- Deallocation (called on)
- De-referencing `operator*`
- Indirection `operator->`
- Assignment `operator=`
- Null Test `operator!` (`operator== 0`)
- Comparison `operator==`, `operator!=`, ...
- Cast `operator(int)`, `operator(T*)`
- Address Of `operator&`
- Address Arithmetic `operator+`, `operator-`, `operator++`, `operator--`, `operator+=`, `operator-=`
- Indexing (array) `operator[]`



What is a Raw Pointer?

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Typical use of Pointers
 - Essential – Link ('[next](#)') in a data structure
 - Inessential – Apparent programming ease
 - ▷ Passing Objects in functions: `void MyFunc(MyClass *)`;
 - ▷ **Smart** expressions: `while (p) cout << *p++`;
- It is not a *First Class Object (FCO)* : An integer value is a FCO
- It Does not have a *Value Semantics* : Cannot COPY or ASSIGN at will
- It is a Weak Semantics for *Ownership* of pointee



Raw Pointers: Ownership Issue of Pointers

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Raw Pointers: Ownership Issue of Pointers



Ownership Issue of Pointers

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- **Ownership Issue – ASSIGN problem**

```
MyClass *p = new MyClass; // Create ownership
p = 0;                      // Lose ownership
```

Memory Leaks!

- **Ownership Issue – COPY problem**

```
MyClass *p = new MyClass; // Create ownership
MyClass *q = p;           // Copy ownership - no Copy Constructor! Performs shallow copy
delete q;                 // Delete Object & Remove ownership
delete p;                 // Delete Object - where is the ownership?
```

Double Deletion Error!

- **Solution Of these: Exception Handling through try-catch**

```
void MyAction() {
    MyClass *p = 0;
    try {
        p = new MyClass;
        p->Function();
    }
    catch (...) {
        delete p; // Repeated code
        throw;
    }
    delete p;
}
```



Pitfall: Handling Ownership Issue of Pointers using try-catch

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Exceptional path dominates regular path

```
void MyDoubleAction() {  
    MyClass *p = 0, *q = 0;  
    try {  
        p = new MyClass;  
        p->Function(); // May throw  
        q = new MyClass;  
        q->Function(); // May throw  
    }  
    catch (...) {  
        delete p; // Repeated code  
        delete q; // Repeated code  
        throw;  
    }  
    delete p;  
    delete q;  
}
```



How to deal with an Object?

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers
Operations

Ownership Issue
Pointers vs.
Reference

Smart Pointers
Policies

Storage Policy
Ownership Policy

Module Summary

So how do we deal with the objects to alleviate such problems?

- The object itself – *by value*
 - Performance Issue
 - Redundancy Issue
- As the memory address of the object – *by pointer*
 - Lifetime Management Issue
 - Code Prone to Memory Errors
- With an alias to the object – *by reference*
 - Good when null-ness is not needed
 - const-ness is often useful



Raw Pointers: Pointers vs. Reference

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

**Pointers vs.
Reference**

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Raw Pointers: Pointers vs. Reference



Pointers Vs. Reference

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Use **Reference** to Objects when
 - Null reference is not needed
 - Reference once created does not need to change
- Avoids
 - The security problems implicit with pointers
 - The (pain of) low level memory management (that is, **delete**)
- Without Pointer – Use Garbage Collection



Smart Pointers

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Smart Pointers

Sources:

- The Rule of The Big Three (and a half) – Resource Management in C++, 2014
- What is a C++ unique pointer and how is it used? smart pointers part I, Soroush Khajepor, 2021
- What is a C++ shared pointer and how is it used? smart pointers part II, Soroush Khajepor, 2021
- What is a C++ weak pointer and where is it used? smart pointers part III, Soroush Khajepor, 2021



What is a Smart Pointer?

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- A Smart pointer is a C++ object
- Stores pointers to dynamically allocated (heap / free store) objects
- Improves raw pointers by implementing
 - Construction & Destruction
 - Copying & Assignment
 - Dereferencing:
 - ▷ `operator->`
 - ▷ unary `operator*`
- Grossly mimics raw pointer syntax and semantics



Typical Tasks of a Smart Pointer

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Selectively *disallows unwanted* operations, that is, Address Arithmetic
- *Lifetime Management*
 - Automatically deletes dynamically created objects at appropriate time
 - On face of exceptions – ensures proper destruction of dynamically created objects
 - Keeps track of dynamically allocated objects shared by multiple owners
- *Concurrency Control*
- Supports **Idioms**: **RAII**: Resource Acquisition is Initialization and **RRID**: Resource Release Is Destruction
 - The idiom makes use of the fact that every time an object is created a constructor is called; and when that object goes out of scope a destructor is called
 - The constructor/destructor pair can be used to create an object that automatically allocates and initialises another object (known as the *managed object*) and cleans up the managed object when it (the *manager*) goes out of scope
 - This mechanism is generically referred to as **resource management**



Typical Smart Pointer Interface

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

```
template <class T> // Pointee type T
class SmartPtr {
public:
    // Constructible
    // No implicit conversion from Raw ptr
    explicit SmartPtr(T* pointee): // RAI
        pointee_(pointee) { }
    // Copy Constructible
    SmartPtr(const SmartPtr& other);
    // Assignable
    SmartPtr& operator=(const SmartPtr& other);
    // Destroys the pointee
    ~SmartPtr() { delete pointee_; } // RRID
    // Dereferencing
    T& operator*() const { ... return *pointee_; }
    // Indirection
    T* operator->() const { ... return pointee_; }
private:
    T* pointee_; // Holding the pointee
};
```



Typical Smart Pointer Use: Mimic a Raw Pointer

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

```
// A class for use
class MyClass {
public:
    void Function();
    // ...
};

// Create a smart pointer as an object
SmartPointer<MyClass> sp(new MyClass); // RAII: sp takes ownership of the instance

// As if indirecting the raw pointer
sp->Function(); // (sp.operator->())->Function()

// As if dereferencing the raw pointer
(*sp).Function();
```



Smart Pointers: Policies

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Smart Pointers: Policies



The Smartness Charter

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- It always points either to a valid allocated object or is NULL
- It deletes the object once there are no more references to it
- Fast: Preferably zero de-referencing and minimal manipulation overhead
- Raw pointers to be only explicitly converted into smart pointers. Easy search using grep is needed (it is unsafe)
- It can be used with existing code
- Programs that do not do low-level stuff can be written exclusively using this pointer. No Raw pointers needed
- Thread-safe
- Exception safe
- It should not have problems with circular references
- *Programmers would not keep raw pointers and smart pointers to the same object*



Policies

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- The charter is managed through a set of policies that bring in flexibility and leads to different flavors of smart pointers
- Major policies include:
 - Storage Policy
 - Ownership Policy
 - Conversion Policy
 - Null-test Policy



Smart Pointers: Policies: Storage Policy

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Smart Pointers: Policies: Storage Policy



3-Way Storage Policy

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- The Storage Type (T^*)
 - The type of pointee: Specialized pointer types possible: FAR, NEAR
 - By *default*, it is a raw pointer
 - Other Smart Pointers possible: When layered
- The Pointer Type (T^*)
 - The type returned by *operator->*
 - ▷ Can be different from the storage type if proxy objects are used
- The Reference Type ($T\&$)
 - The type returned by *operator**



Smart Pointers: Policies: Ownership Policy

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Smart Pointers: Policies: Ownership Policy



Ownership Management Policy

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Smart pointers are about *ownership of pointees*
- **Exclusive Ownership**
 - Every smart pointer has an exclusive ownership of the pointee
 - `std::unique_ptr`
 - Use **Destructive Copy**
- **Shared Ownership**
 - Ownership of the pointee is shared between Smart pointers – more than one smart pointer holds the same pointee
 - `std::shared_ptr`
 - `std::weak_ptr`
 - Track the Smart pointer references for lifetime
 - ▷ **Reference Counting**
 - ▷ **Reference Linking**



Ownership Policy: Destructive Copy

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- **Exclusive Ownership Policy**
- Transfer ownership on copy
- Source Smart Pointer in a copy is set to `NULL` / `nullptr`
- Available in C++ Standard Library
 - `std::unique_ptr`
- Implemented in
 - Copy Constructor
 - `operator=`



Ownership Policy: Destructive Copy

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

```
template <class T>
class SmartPtr { public:
    SmartPtr(SmartPtr& src) { // Src ptr is not const
        pointee_ = src.pointee_; // Copy
        src.pointee_ = 0;        // Remove ownership for src ptr
    }
    SmartPtr& operator=(SmartPtr& src) { // Src ptr is not const
        if (this != &src) {           // Check & skip self-copy
            delete pointee_;           // Release destination object
            pointee_ = src.pointee_;   // Assignment
            src.pointee_ = 0;          // Remove ownership for src ptr
        }
        return *this; // Return the assigned Smart Pointer
    } // ...
};
```

- Note that the copy operations (**lvalue** binding) here actually moves the resource – transfers ownership. Hence, the source object needs a non-const reference
- Though the semantics is similar to move operations, these are different from move operators (**rvalue** binding) due to **lvalue** binding



Ownership Policy: Destructive Copy: The Maelstrom Effect

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Consider a call-by-value

```
void Display(SmartPtr<Something> sp); // passed by value - copy performed
SmartPtr<Something> sp(new Something);
Display(sp); // sinks sp
```

- Display acts like a maelstrom of smart pointers:
 - It sinks any smart pointer passed to it
 - After `Display(sp)` is called, `sp` holds the null pointer
- Lesson: **Pass Smart Pointers by Reference**
- Smart pointers with destructive copy cannot usually be stored in containers and in general must be handled with care
- **STL Containers need FCO**



Ownership Policy: Destructive Copy: Advantages

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Incurs almost no overhead.
- Good at enforcing ownership transfer semantics
 - Use the *maelstrom effect* to ensure that the function takes over the passed-in pointer
- Good as return values from functions
 - The pointee object gets destroyed if the caller does not use the return value
- Excellent as stack variables in functions that have multiple return paths
- Available in Standard Library
 - `std::auto_ptr` [`C++03`, deprecated in `C++11`, removed in `C++17`]
 - `std::unique_ptr` [`C++11`]



Ownership Policy: Reference Counting / Linking

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- **Shared Ownership Policy**
- Allow multiple Smart pointers to point to the same pointee
- **Reference Counting**: A count of the number of Smart pointers (references) to a pointee
 - *Non-Intrusive Counter*: Multiple / Single Raw Pointers per pointee with count in free store
 - *Intrusive Counter*: Count is a member of the object
 - Destroy the pointee Object when the count equals 0
- **Reference Linking**: All Smart pointers to a pointee are *linked on a chain*
 - The exact count is not maintained – only check if the chain is null
 - Destroy the pointee Object when the chain gets empty
- Available in C++ Standard Library
 - `std::shared_ptr`
 - `std::weak_ptr`
- Implemented in
 - Constructor
 - Copy Constructor
 - `operator=`
 - Destructor



Ownership Policy: Exclusive and Shared

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

Exclusive Ownership



- **Exclusive Ownership Policy**
- Transfer ownership on copy
- On Copy: Source is set to NULL
- On Delete: Destroy the pointee Object
- `std::auto_ptr` (C++03), `std::unique_ptr` (C++11)
- Coded in: C-Ctor, operator=

Shared Ownership



- **Shared Ownership Policy**
- Multiple Smart pointers to same pointee
- On Copy: Reference Count (RC) incremented
- On Delete: RC decremented, if RC > 0. Pointee object destroyed for RC = 0
- `std::shared_ptr`, `std::weak_ptr` (C++11)
- Coded in: Ctor, C-Ctor, operator=, Dtor



Ownership Policy: Exclusive and Shared

Module M56

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs. Reference

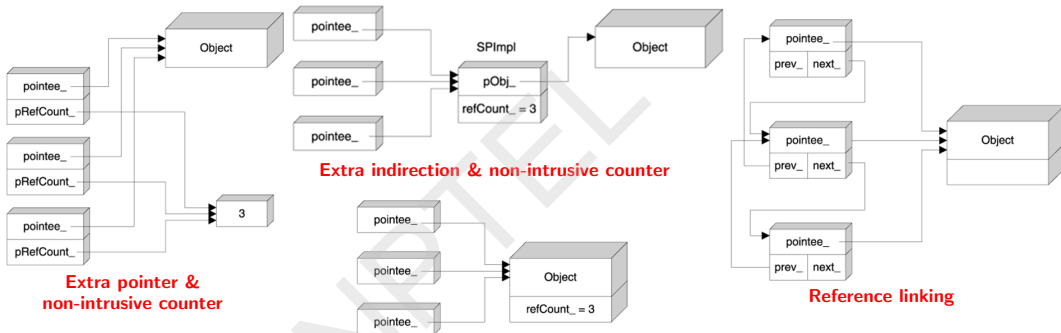
Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary



● Non-Intrusive Counter

- Addl. count ptr per smart ptr
- Count in Free Store
- Allocation of Count may be slow as it is too small (may be improved by global pool)

● Non-Intrusive Counter

- Addl. count ptr removed
- But addl. access level means slower speed

● Intrusive Counter

- Most optimized RC smart ptr
- Cannot work for an already existing design
- Used in Component Object Model (COM)

● Reference Linking

- Overhead of two addl. ptrs
- Doubly-linked list for constant time:
 - ▷ For Append, Remove & Empty detection



Module Summary

Module M56

Partha Pratim
Das

Weekly Recap

Objectives &
Outlines

Raw Pointers

Operations

Ownership Issue

Pointers vs.
Reference

Smart Pointers

Policies

Storage Policy

Ownership Policy

Module Summary

- Revisited Raw Pointers and discussed how to deal with the objects through raw pointer
- Introduced Smart pointers with typical interface and use
- Introduced some of the policies for smart pointer:
 - Storage Policies
 - Ownership Policies



Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Programming in Modern C++

Module M57: C++11 and beyond: Resource Management by Smart Pointers: Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M57

Partha Pratim
Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Revisited Raw Pointers and discussed how to deal with the objects through raw pointer
- Introduced Smart pointers with typical interface and use
- Introduced some of the policies for smart pointer:
 - Different storage policies
 - Ownership Policies



Module Objectives

Module M57

Partha Pratim
Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- To continue Discussions on various policies of smart pointers
 - Ownership Policies
 - Implicit Conversion policy
 - Null test policy
- To familiarize with Resource Management using Smart Pointers from Standard Library
 - `unique_ptr`
 - `shared_ptr`
 - `weak_ptr`
 - `auto_ptr`



Module Outline

Module M57

Partha Pratim
Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- 1 Smart Pointers
 - Recap
 - Ownership Policy
 - Conversion Policy
 - Null-test Policy
- 2 Resource Management
 - `std::unique_ptr`
 - `std::shared_ptr`
 - `std::weak_ptr`
 - `std::auto_ptr`
 - Summary of Smart Pointer Operations
 - Binary Tree
- 3 Recommendations for Smart Pointers
- 4 Module Summary



Smart Pointers

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Sources:

- Chapter 4. Smart Pointers: Effective Modern C++, Scott Meyers
 - Item 18: Use `std::unique_ptr` for exclusive-ownership resource management
 - Item 19: Use `std::shared_ptr` for shared-ownership resource management
 - Item 20: Use `std::weak_ptr` for `std::shared_ptr`-like pointers that can dangle
 - Item 21: Prefer `std::make_unique` and `std::make_shared` to direct use of `new`
- The Rule of The Big Three (and a half) – Resource Management in C++, 2014
- Smart pointer, wikipedia
- How to use C++ raw pointers properly?, Soroush Khajepor, 2020
- What is a C++ unique pointer and how is it used? smart pointers part I, Soroush Khajepor, 2021
- What is a C++ shared pointer and how is it used? smart pointers part II, Soroush Khajepor, 2021
- What is a C++ weak pointer and where is it used? smart pointers part III, Soroush Khajepor, 2021
- Lambdas: Smart Pointers, Jim Fix, Reed College

Smart Pointers



Smart Pointers: Recap

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Smart Pointers: Recap



What is a Smart Pointer? (Recap Module 56)

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- A Smart pointer is a C++ object
- Stores pointers to dynamically allocated (heap / free store) objects
- Improves raw pointers by implementing
 - Construction & Destruction
 - Copying & Assignment
 - Dereferencing:
 - ▷ `operator->`
 - ▷ unary `operator*`
- Grossly mimics raw pointer syntax and semantics



Typical Tasks of a Smart Pointer (Recap Module 56)

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Selectively *disallows unwanted* operations, that is, Address Arithmetic
- *Lifetime Management*
 - Automatically deletes dynamically created objects at appropriate time
 - On face of exceptions – ensures proper destruction of dynamically created objects
 - Keeps track of dynamically allocated objects shared by multiple owners
- *Concurrency Control*
- Supports **Idioms**: **RAII**: Resource Acquisition is Initialization Idiom and **RRID**: Resource Release Is Destruction
 - The idiom makes use of the fact that every time an object is created a constructor is called; and when that object goes out of scope a destructor is called
 - The constructor/destructor pair can be used to create an object that automatically allocates and initialises another object (known as the *managed object*) and cleans up the managed object when it (the *manager*) goes out of scope
 - This mechanism is generically referred to as **resource management**



Typical Smart Pointer Interface (Recap Module 56)

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
template<typename T> // Pointee type T
class SmartPtr {
public:
    // Constructible
    // No implicit conversion from Raw ptr
    explicit SmartPtr(T* pointee):
        pointee_(pointee) { }
    // Copy Constructible
    SmartPtr(const SmartPtr& other);
    // Assignable
    SmartPtr& operator=(const SmartPtr& other);
    // Destroys the pointee
    ~SmartPtr();
    // Dereferencing
    T& operator*() const { ... return *pointee_; }
    // Indirection
    T* operator->() const { ... return pointee_; }
private:
    T* pointee_; // Holding the pointee
};
```



The Smartness Charter (Recap Module 56)

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- It always points either to a valid allocated object or is NULL
- It deletes the object once there are no more references to it
- Fast: Preferably zero de-referencing and minimal manipulation overhead
- Raw pointers to be only explicitly converted into smart pointers. Easy search using grep is needed (it is unsafe)
- It can be used with existing code
- Programs that do not do low-level stuff can be written exclusively using this pointer. No Raw pointers needed
- Thread-safe
- Exception safe
- It should not have problems with circular references
- *Programmers would not keep raw pointers and smart pointers to the same object*



Policies (Recap Module 56)

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- The charter is managed through a set of policies that bring in flexibility and leads to different flavors of smart pointers
- Major policies include:
 - Storage Policy
 - Ownership Policy
 - Conversion Policy
 - Null-test Policy



Ownership Policy: Exclusive and Shared (Recap Module 56)

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Exclusive Ownership



- **Exclusive Ownership Policy**
- Transfer ownership on copy
- On Copy: Source is set to NULL
- On Delete: Destroy the pointee Object
- `std::auto_ptr` (C++03), `std::unique_ptr` (C++11)
- Coded in: C-Ctor, operator=

Shared Ownership



- **Shared Ownership Policy**
- Multiple Smart pointers to same pointee
- On Copy: Reference Count (RC) incremented
- On Delete: RC decremented, if RC > 0. Pointee object destroyed for RC = 0
- `std::shared_ptr`, `std::weak_ptr` (C++11)
- Coded in: Ctor, C-Ctor, operator=, Dtor



Ownership Policy: Exclusive and Shared (Recap Module 56)

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

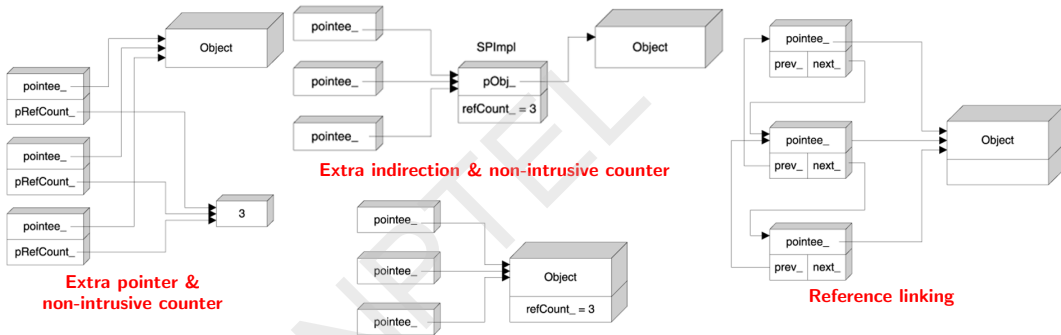
`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary



● Non-Intrusive Counter

- Addl. count ptr per smart ptr
- Count in Free Store
- Allocation of Count may be slow as it is too small (may be improved by global pool)

● Non-Intrusive Counter

- Addl. count ptr removed
- But addl. access level means slower speed

● Intrusive Counter

- Most optimized RC smart ptr
- Cannot work for an already existing design
- Used in Component Object Model (COM)

● Reference Linking

- Overhead of two addl. ptrs
- Doubly-linked list for constant time:
 - ▷ For Append, Remove & Empty detection



Smart Pointers: Ownership Policy

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Smart Pointers: Ownership Policy



Ownership Policy: Reference Management: Shortcoming

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Circular / Cyclic Reference

- Object A holds a smart pointer to an object B. Object B holds a smart pointer to A. Forms a cyclic reference
 - ▷ Typical for a Tree: Child & Parent pointers
- Cyclic references go undetected
 - ▷ Both the two objects remain allocated forever
 - ▷ Resource Leak occurs
- The cycles can span multiple objects



Ownership Policy: Cyclic Reference: Hack

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

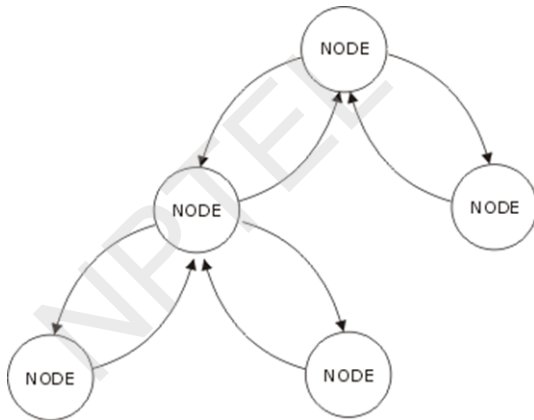
`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary



- Use *Smart pointer* (`std::shared_ptr`) from Parent to Child: *Data Structure* Pointers
- Use *Weak pointer* (`std::weak_ptr`) from Child to Parent: *Algorithm* Pointers



Ownership Policy: Cyclic Reference: Solution

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Maintain two flavors of RC Smart Pointers
 - **Strong** pointers that really link up the data structure (Child / Sibling Links). They behave like regular RC. `std::shared_ptr`
 - ▷ These are **Ownership** pointers
 - **Weak** pointer for cross / back references in the data structure (Parent / Reverse Sibling Links). `std::weak_ptr`
 - ▷ These are **Observer** pointers
- Keep two reference counts:
 - One for strong pointers, and
 - One for weak pointers
- While dereferencing a weak pointer, check the strong reference count:
 - If it is zero, return NULL. As if, the object is gone



Smart Pointers: Conversion Policy

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Smart Pointers: Conversion Policy



Implicit Conversion Policy

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Consider

```
void Fun(Something* p); ... // For maximum compatibility this should work
SmartPtr<Something> sp(new Something);
Fun(sp); // OK or error?
```

- User-Defined Conversion (cast)

```
template<typename T>class SmartPtr { public:
    operator T*() { return pointee_; } // user-defined conversion to T*
};
```

- Pitfall:** This following compiles okay and defeats the purpose of the smart pointer

```
SmartPtr<Something> sp; ... // Undetected semantic error at compile time
delete sp; // Compiler passes this by casting to raw pointer
```

- No conversion allowed in library.** No `operator T*() const noexcept;` is even provided. Use `get()` to obtain the raw pointer from `unique_ptr` or `shared_ptr`



Smart Pointers: Null-test Policy

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Smart Pointers: Null-test Policy



Null Test Policy

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- How to check if the smart pointer is null? Expect the following to work?

```
SmartPtr<Something> sp1, sp2;  
Something* p; ...
```

```
if (sp1)           // Test 1: direct test for non-null pointer ...  
if (!sp1)          // Test 2: direct test for null pointer ...  
if (sp1 == 0)      // Test 3: explicit test for null pointer ...
```

- Without implicit conversion to raw pointers, these cannot work
- Overloading `bool operator!() { return pointee_ == 0; }` would pass **Test 2**, would need **Test 1** to be written as `if (!!sp)`, and fail **Test 3**
- The library provides `explicit operator bool() const noexcept;` for the purpose in `unique_ptr` and `shared_ptr`
- **Test 1**, **Test 2** and **Test 3** work



Resource Management

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Sources:

- Chapter 4. Smart Pointers: Effective Modern C++, Scott Meyers
 - Item 18: Use `std::unique_ptr` for exclusive-ownership resource management
 - Item 19: Use `std::shared_ptr` for shared-ownership resource management
 - Item 20: Use `std::weak_ptr` for `std::shared_ptr`-like pointers that can dangle
 - Item 21: Prefer `std::make_unique` and `std::make_shared` to direct use of `new`
- Smart Pointer in C++ Standard Library
 - `std::unique_ptr`, cppreference
 - `std::shared_ptr`, cppreference
 - `std::weak_ptr`, cppreference
 - `std::auto_ptr`, cppreference
- The Rule of The Big Three (and a half) – Resource Management in C++, 2014

Resource Management



Resource Management

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers
Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Smart pointers enable automatic, exception-safe, object lifetime management
- The various Pointers are:
 - `unique_ptr`: smart pointer with unique object ownership semantics
 - `shared_ptr`: smart pointer with shared object ownership semantics
 - `weak_ptr`: weak reference to an object managed by `std::shared_ptr`
 - `auto_ptr`: smart pointer with strict object ownership semantics
- All these are Defined in header `<memory>`
- First three pointers are included in C++11 where as last one is as in C++03



Resource Management: `std::unique_ptr`

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: `std::unique_ptr`

Sources:

- `std::unique_ptr`, [cppreference](#)



std::unique_ptr

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
// Managing a single object. Deleter may be use supplied or default delete
template<typename T, typename Deleter = std::default_delete <T> >
class unique_ptr;
```

```
// Managing an array of object
template<typename T, typename Deleter>
class unique_ptr<T[], Deleter>;
```

- It retains sole ownership of an object through a pointer and destroys that object when the `unique_ptr` goes out of scope
- No two `unique_ptr` instances can manage the same object
- The raw pointer to the managed object can be obtained by `get()`
- The object is destroyed and its memory deallocated when:
 - The managing `unique_ptr` object is destroyed, or
 - The managing `unique_ptr` object is assigned another pointer via `operator=` or `reset()`
- The ownership can also be relinquished by `release()` which returns the raw pointer of the managed object



std::unique_ptr

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

- The object is destroyed using a potentially user-supplied deleter by calling `Deleter(ptr)`
- A `unique_ptr` may alternatively own no object (managed object pointer is `nullptr`), in which case it is called *empty*
- There are two versions of `std::unique_ptr`:
 - Manages the lifetime of a single object (for example, allocated with `new`)
 - Manages the lifetime of a dynamically-allocated array of objects (for example, allocated with `new[]`)
- Typical uses of `std::unique_ptr` include:
 - exception safety to classes and functions that handle objects with dynamic lifetime, by guaranteeing deletion
 - ownership of uniquely-owned objects with dynamic lifetime into functions
 - ownership of uniquely-owned objects with dynamic lifetime from functions
 - element type in move-aware containers, such as `std::vector`.



std::unique_ptr: Example

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
#include <iostream>
#include <memory>

struct Foo {
    Foo() { std::cout << "Foo::Foo\n"; }
    ~Foo() { std::cout << "Foo::~~Foo\n"; }
    void bar() { std::cout << "Foo::bar\n"; }
};

void f(const Foo &) { std::cout << "f(const Foo&)\n"; }

int main() {
    std::unique_ptr<Foo> p1 = std::make_unique<Foo>(); // (C++14) p1 owns Foo. // Foo::Foo
    // std::unique_ptr<Foo> p1(new Foo); // (C++11) p1 owns Foo. // Foo::Foo
    if (p1) p1->bar(); // Foo::bar
    {
        std::unique_ptr<Foo> p2(std::move(p1)); // now p2 owns Foo
        f(*p2); // f(const Foo&)
        p1 = std::move(p2); // ownership returns to p1
        std::cout << "destroying p2...\n"; // destroying p2...
    }
    if (p1) p1->bar(); // Foo instance is destroyed when p1 goes out of scope. // Foo::bar
} // Foo::~~Foo
```



Resource Management: `std::shared_ptr`

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: `std::shared_ptr`

Sources:

- [`std::shared\_ptr`](#), `cppreference`



std::shared_ptr

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
template<typename T> class shared_ptr;
```

- It retains shared ownership of an object through a pointer. Several `shared_ptr` objects may own the same object
- Object is destroyed and its memory deallocated when either of the following happens:
 - the last remaining `shared_ptr` owning the object is destroyed
 - the last remaining `shared_ptr` owning the object is assigned another pointer via `operator=` or `reset()`
- The object is destroyed using delete-expression or a custom deleter that is supplied to `shared_ptr` during construction
- The raw pointer to the managed object can be obtained by `get()`
- We can get the number of managed objects by invoking `use_count()`
- A `shared_ptr` can share ownership of an object while storing a pointer to another object
- It may also own no objects, in which case it is called *empty*
- All specializations of `shared_ptr` meet the requirements of `CopyConstructible`, `CopyAssignable`, and `LessThanComparable` and are contextually convertible to `bool`



std::shared_ptr: Example

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
#include <iostream>
#include <memory> // We need to include this for shared_ptr
using namespace std;
int main() {
    shared_ptr<int> p1 = make_shared<int>(); // Creating through make_shared
    *p1 = 78; // Set a value for the managed object
    cout << "p1 = " << *p1 << endl; // Access the value from managed object: p1 = 78
    cout << "p1 RC = " << p1.use_count() << endl; // Show RC: p1 RC = 1
    shared_ptr<int> p2(p1); // Second shared_ptr points to same pointer. RC = 2
    cout << "p2 RC = " << p2.use_count() << endl; // Show RC: p2 RC = 2
    cout << "p1 RC = " << p1.use_count() << endl; // Show RC: p1 RC = 2
    if (p1 == p2) { cout << "Same objects\n"; } // Compare ptrs: Same object
    cout << "Reset p1 " << endl; // : Reset p1
    p1.reset(); // Reset the shared_ptr - it will not point to any object
    cout << "p1 RC = " << p1.use_count() << endl; // RC = 0: p1 RC = 0
    p1.reset(new int(11)); // Reset the shared_ptr with a new Pointer
    cout << "p1 RC = " << p1.use_count() << endl; // RC = 1: p1 RC = 1
    p1 = nullptr; // Assign nullptr to de-attach managed object
    cout << "p1 RC = " << p1.use_count() << endl; // RC = 0: p1 RC = 0
    if (!p1) { cout << "p1 is NULL" << endl; } // Test pointer: p1 is NULL
}
```



Resource Management: `std::weak_ptr`

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: `std::weak_ptr`

Sources:

- [`std::weak\_ptr`](#), `cppreference`



std::weak_ptr

Module M57

Partha Pratim Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
template<typename T> class weak_ptr;
```

- It holds a non-owning (*weak*) reference to an object that is managed by `std::shared_ptr`
- It must be converted to `std::shared_ptr` in order to access the referenced object
- `std::weak_ptr` models temporary ownership: when an object needs to be accessed only if it exists, and it may be deleted at any time by someone else
- It is used to track the object, and it is converted to `std::shared_ptr` to assume temporary ownership
- We can get the number of managed objects by invoking `use_count()`
- To check if the managed object is already deleted we can call `expired()`
- Also, `lock()` can be used to create a `shared_ptr` from a `weak_ptr` to manage the referenced object
- If the original `std::shared_ptr` is destroyed at this time, the object's lifetime is extended until the temporary `std::shared_ptr` is destroyed as well
- **Note:** It is used to break circular references of `std::shared_ptr`. It cannot be used to access the managed object



std::weak_ptr: Example

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

```
#include <iostream>
```

```
#include <memory>
```

```
std::weak_ptr<int> gw;
```

```
void f() {
```

```
    if (auto spt = gw.lock()) { // Has to be copied into a shared_ptr before usage
```

```
        std::cout << *spt << "\n";
```

```
    }
```

```
    else { std::cout << "gw is expired\n"; }
```

```
}
```

```
int main() {
```

```
{
```

```
    auto sp = std::make_shared<int>(42); //
```

```
    gw = sp;
```

```
    f(); // 42
```

```
}
```

```
    f(); // gw is expired
```

```
}
```



Resource Management: `std::auto_ptr`

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: `std::auto_ptr`

Sources:

- `std::auto_ptr`, `cppreference`



std::auto_ptr

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers
Recap

Ownership Policy
Conversion Policy
Null-test Policy

Resource
Management

std::unique_ptr
std::shared_ptr
std::weak_ptr
std::auto_ptr
Summary

Binary Tree

Recommendations

Module Summary

```
// Managing a single object in C++03. Deprecated in C++11, removed in C++17  
template<typename T> class auto_ptr;
```

```
template<> class auto_ptr<void>;
```

- It retains sole ownership of an object through a pointer and destroys that object when the `auto_ptr` goes out of scope
- No two `auto_ptr` instances can manage the same object
- The raw pointer to the managed object can be obtained by `get()`
- The object is destroyed and its memory deallocated when:
 - The managing `auto_ptr` object is destroyed, or
 - The managing `auto_ptr` object is assigned another pointer via `operator=` or `reset()`
- The ownership can also be relinquished by `release()` which returns the raw pointer of the managed object
- **Never use `auto_ptr` in C++11 and beyond**



Resource Management: Summary of Smart Pointer Operations

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: Summary of Smart Pointer Operations



Summary of Smart Pointer Operations

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

std::unique_ptr

std::shared_ptr

std::weak_ptr

std::auto_ptr

Summary

Binary Tree

Recommendations

Module Summary

Member	unique_ptr	shared_ptr	weak_ptr	auto_ptr	Remarks
operator=	Y	Y	Y	Y	assigns the ptr ¹
release	Y	N	N	Y	returns a ptr to the managed object and releases the ownership
reset	Y	Y	Y	Y	replaces the managed object
swap	Y	Y	Y	N	swaps the managed objects
get	Y	Y	N	Y	returns a ptr to the managed obj
operator bool	Y	Y	N	N	checks if the stored ptr is not null
owner_before	N	Y	Y	N	owner-based ordering of smart pointers
operator*	Y	Y	N	Y	accesses the managed object
operator->	Y	Y	N	Y	accesses the managed object
operator[]	Y	Y (C++17)	N	N	indexed access to the managed array
use_count	N	Y	Y	N	returns the number of shared_ptr objects that manage the object
make_unique (C++14)	unique_ptr			creates a unique ptr that manages a new object creates a shared pointer that manages a new object applies static_cast to the stored ptr applies dynamic_cast to the stored ptr applies const_cast to the stored ptr applies reinterpret_cast to the stored ptr (C++17) checks whether the referenced obj was already deleted creates a shared_ptr that manages the referenced object	
make_shared	shared_ptr				
static_pointer_cast	shared_ptr				
dynamic_pointer_cast	shared_ptr				
const_pointer_cast	shared_ptr				
reinterpret_pointer_cast	shared_ptr				
expired	weak_ptr				
lock	weak_ptr				

¹transfers ownership from another `auto_ptr`



Resource Management: Binary Tree

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Resource Management: Binary Tree

Sources:

- `std::shared_ptr`, cppreference
- `std::weak_ptr`, cppreference



Binary Tree

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers
Recap

Ownership Policy
Conversion Policy
Null-test Policy

Resource
Management

`std::unique_ptr`
`std::shared_ptr`
`std::weak_ptr`
`std::auto_ptr`
Summary

Binary Tree

Recommendations

Module Summary

- We show an example of a binary tree where every node keep a back pointer to its parent
- This leads to circularity and using `std::shared_ptr` we cannot clean up the tree
- So we use `std::shared_ptr` for the two children and `std::weak_ptr`
- Similar strategy may be employed in every case of circular data structure design
- Note that using `std::shared_ptr` for a binary tree may be an overkill as every node is held by its unique parent. So using `std::unique_ptr` for child and raw pointer for parent may be more optimal



Binary Tree using `std::shared_ptr` and `std::weak_ptr`

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

```
#include <iostream>
#include <memory>
using namespace std;
struct Node {
    shared_ptr<Node> lc;    // owns left child
    shared_ptr<Node> rc;    // owns right child
    weak_ptr<Node> parent; // observes parent
    int v;                  // Node value
    Node(int i = 0): v(i)
    { cout << "Node = " << v << endl; }
    ~Node()
    { cout << "~Node = " << v << endl; }
};
```

Node = 2

Node = 1

Node = 3

2 1 3

~Node = 2 // Nodes will not be cleaned

~Node = 3 // if parent is a shared_ptr

~Node = 1 // This is due to circularity

```
int main() {
    shared_ptr<Node> root = // root: 2
        make_shared<Node>(2);
    root->lc = // left child: 1
        make_shared<Node>(1);
    root->rc = // right child: 3
        make_shared<Node>(3);
    root->lc->parent = root; // back link
    root->rc->parent = root;

    shared_ptr<Node> p = root; // visit tree
    weak_ptr<Node> q;          // hold parent
    cout << p->v << ' ';
    p = p->lc;
    cout << p->v << ' ';
    q = p->parent;
    p = q.lock(); // weak to shared
    p = p->rc;
    cout << p->v << ' ';
    cout << endl;
}
```



Recommendations for Smart Pointers

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

Recommendations for Smart Pointers



Recommendations for Smart Pointers

Module M57

Partha Pratim Das

Objectives & Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource

Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Scott Meyers in his book [Effective Modern C++](#) (Chapter 4. Smart Pointers) has made the following recommendations for the use of smart pointers for resource management:
 - Item 18: Use `std::unique_ptr` for exclusive-ownership resource management
 - Item 19: Use `std::shared_ptr` for shared-ownership resource management
 - Item 20: Use `std::weak_ptr` for `std::shared_ptr`-like pointers that can dangle
 - Item 21: Prefer `std::make_unique` and `std::make_shared` to direct use of `new`
- We strongly recommend the use of these for modern designs



Module Summary

Module M57

Partha Pratim
Das

Objectives &
Outlines

Smart Pointers

Recap

Ownership Policy

Conversion Policy

Null-test Policy

Resource
Management

`std::unique_ptr`

`std::shared_ptr`

`std::weak_ptr`

`std::auto_ptr`

Summary

Binary Tree

Recommendations

Module Summary

- Discussed various policies of smart pointer
 - Ownership Policies
 - Implicit Conversion policy
 - Null test policy
- Familiarized with Resource Management using Smart Pointers from Standard Library
 - `unique_ptr`
 - `shared_ptr`
 - `weak_ptr`
 - `auto_ptr`



Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

Programming in Modern C++

Module M58: C++11 and beyond: Concurrency: Part 1

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

- Discussed various policies of smart pointer
 - Ownership Policies
 - Implicit Conversion policy
 - Null test policy
- Familiarized with Resource Management using Smart Pointers from Standard Library
 - `unique_ptr`
 - `shared_ptr`
 - `weak_ptr`
 - `auto_ptr`



Module Objectives

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

- To introduce the notion of concurrent programming in C++11 using thread support
- To explore library support through `std::thread` and `std::bind`
- To expose to the bugs in thread programming - race condition and data race
- To discuss examples of thread programs with bugs and their solution



Module Outline

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

1 thread Programming in C++

- `std::thread`
- `std::bind`

2 Race Condition & Data Race

- Race Condition Example
 - Solution by Mutex
 - Solution by Atomic

3 Module Summary



thread Programming in C++

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

Sources:

- C++11 - the new ISO C++ standard: Threads, Stroustrup, 2016
- C++11 Standard Library Extensions — Concurrency: Threads, isocpp
- `std::thread`, cppreference
- Concurrency memory model, isocpp.org
- An Overview of the New C++ (C++11/14), Scott Meyers Training Courses
- A tutorial on modern multithreading and concurrency in C++, 2020
- C++11 Multi-threading Tutorials: Parts 1-8, [thisPointer](#)
 - C++11 Multithreading – Part 1 : Three Different ways to Create Threads
- C++20 Concurrency: Parts 1-3, Gajendra Gulgia, 2021
 - C++20 Concurrency: Part 1: synchronized output stream
 - C++20 Concurrency: Part 2: `jthreads`
 - C++20 Concurrency: Part 3: `request_stop` and `stop_token` for `std::jthread`

thread Programming in C++



Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Unlike built-in multi-threading support for Java, C / C++ needs **libraries for multi-threading**
- Typically features of a Multi-threading Library include:
 - Thread Management: Create, Join threads etc.
 - Synchronization: Mutex, Lock, Condition variables, Barrier, Future etc.
 - Thread Local Storage
- Third-Party Library
 - **POSIX Threads** (aka pthreads), is an execution model that exists independently from a language, as well as a parallel execution model
 - **Boost C++ Threads**
 - **Multithreading with C and Win32, Multithreading with C++ and MFC** by Microsoft
 - Others: **OpenMP, OpenThreads, Qt QThread, Parallel Pattern Library, oneAPI Threading Building Blocks (oneTBB), IPP**, etc.
- Language Provided Library
 - **Concurrency Support Library for C: C11 / C17**
 - **Concurrency Support Library for C++: C++11 / C++14 / C++17 / C++20**
 - ▷ We will discuss here



Spawn Thread

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- A **thread** is a representation of an *execution/computation in a program*
- In C++11, a thread *usually shares an address space with other threads* – it differs from a process, which generally does not directly share data with other processes
- C++ has had a host of threads implementations for a variety of hardware and operating systems in the past, *what is new is a standard-library threads library*
- A thread is launched by constructing a `std::thread` with a function / a function object / a λ :

```
#include <iostream>
#include <thread>
using namespace std;

void f() { cout << "In f()" << endl; };
struct F { void operator()() { cout << "In F()()" << endl; }; };
int main() {
    std::thread t1{f};          // f() executes in separate thread
    std::thread t2{F()};        // F()() executes in separate thread
}
// terminate called without an active exception
```

- This program is unlikely to give any useful results – whatever `f()` and `F()` might do
- The program may terminate before or after `t1` executes `f()` and before or after `t2` executes `F()`



Join Thread

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread

std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

- We need to wait for the two tasks to complete:

```
#include <iostream>
#include <thread>
using namespace std;
void f() { cout << "In f()" << endl; };
struct F { void operator()() { cout << "In F()()" << endl; }; };
int main() {
    std::thread t1{f};           // f() executes in separate thread
    std::thread t2{F{}};         // F()() executes in separate thread

    t1.join(); // wait for t1
    t2.join(); // wait for t2
}
In f() // 10 out of 15 attempts outputs this
In F()()

// Non-deterministic behavior on display

In F()() // 5 out of 15 attempts outputs this
In f()
```

- The `join()`s ensure that we don't terminate until the threads have completed. *To join means to wait for the thread to terminate*



Thread with Parameters

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread

std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Typically, we would like to pass some arguments to the task to be executed. For example:

```
#include <iostream>
#include <thread>
#include <vector>
#include <functional> // std::bind
using namespace std;

void f(vector<int>& v) { cout << "In f()" << ": ";
    for(auto x : v) { cout << ' ' << x; } cout << endl;
};

struct F { vector<int>& v;
    F(vector<int>& vv) :v{vv} { }
    void operator() { cout << "In F()" << ": ";
        for(auto x : v) { cout << ' ' << x; } cout << endl;
    };
};

int main() {
    vector<int> my_vec {2, 3, 5, 7, 11 }; // Init vector
    std::thread t1{std::bind(f, my_vec)}; // f(my_vec) executes in separate thread
    std::thread t2{F(my_vec)};           // F(my_vec)() executes in separate thread
    t1.join(); t2.join();
}

In F():  2 3 5 7 11 // In f():  2 3 5 7 11In F()
In f():  2 3 5 7 11 // :  2 3 5 7 11
```

- Basically, the standard library function `std::bind` makes a function object of its arguments



Thread with Output

- In general, we would also like to get a result back from an executed task
- With plain tasks, there is no notion of a return value; `std::future` is the correct default choice for that
- Alternatively, we can pass an argument to a task telling it where to put its result. For example:

```
#include <iostream>
#include <thread>
#include <vector>
#include <functional> // std::bind
using namespace std;
void f(vector<int>& v, int* res) { cout << "In f()" << ": "; *res = 0; // Function with output
    for(auto x : v) { cout << ' ' << x; *res += x; } cout << endl; // Accumulate sum
};
struct F { vector<int>& v; int* res; // Functor with output
    F(vector<int>& v, int* res) : v{v}, res{res} { *res = 0; }
    void operator()() { cout << "In F()()" << ": ";
        for(auto x : v) { cout << ' ' << x; *res += x; } cout << endl; // Accumulate sum
    };
};
int main() { vector<int> my_vec {2, 3, 5, 7, 11 }; int res1, res2;
    std::thread t1{std::bind(f, my_vec, &res1)}; // f(my_vec) executes in separate thread
    std::thread t2{F(my_vec, &res2)};           // F(my_vec)() executes in separate thread
    t1.join(); t2.join(); std::cout << res1 << ' ' << res2 << '\n';
}
In f():  2 3 5 7 11
In F()(): 2 3 5 7 11
```




thread Programming in C++: `std::thread`

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`

`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

thread Programming in C++: `std::thread`

Sources:

- C++11 Standard Library Extensions — Concurrency: Threads, isocpp
- `std::thread`, cppreference
- A tutorial on modern multithreading and concurrency in C++, 2020
- C++11 Multithreading – Part 1 : Three Different ways to Create Threads



std::thread

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread

std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

```
class thread;
```

- Defined in `<thread>`
- The `class thread` represents a *single thread of execution*. Threads allow *multiple functions to execute concurrently*
- Threads *begin execution immediately upon construction of the associated thread object* (pending any OS scheduling delays), *starting at the top-level function* provided as a constructor argument
- The *return value of the top-level function is ignored* and if it terminates by throwing an exception, `std::terminate` is called
- The top-level function may communicate its return value or an exception to the caller via `std::promise` or by modifying shared variables (which may require synchronization, see `std::mutex` and `std::atomic` later)
- `std::thread` objects may also be in the state that does not represent any thread (after default construction, move from, `detach`, or `join`), and a thread of execution may not be associated with any thread objects (after `detach`)
- No two `std::thread` objects may represent the same thread of execution:
 - `std::thread` is not `CopyConstructible` or `CopyAssignable`
 - `std::thread` is `MoveConstructible` and `MoveAssignable`



std::thread::id and std::thread::thread

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

- `id`: represents the id of a thread

- Member Functions:

- `std::thread::thread`: Constructor: Constructs the thread object

```
// A new thread object which does not represent a thread
```

```
thread() noexcept;
```

```
// Move constructor. Constructs the thread object to represent the thread of  
// execution that was represented by other. After this call other no longer  
// represents a thread of execution
```

```
thread(thread&& other) noexcept;
```

```
// Creates new std::thread object and associates it with a thread of execution
```

```
// The new execution starts with (std::move(f_copy), std::move(args_copy)...)
```

```
template<class Function, class... Args>
```

```
explicit thread(Function&& f, Args&&... args);
```

```
// The copy constructor is deleted; threads are not copyable. No two std::thread  
// objects may represent the same thread of execution
```

```
thread(const thread&) = delete;
```



`std::thread::~~thread` and `std::thread::operator=`

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Member Functions:

- `std::thread::~~thread()`: Destructor: Destroys the thread object
 - ▷ If `*this` has an associated thread (`joinable() == true`), `std::terminate()` is called
 - ▷ A thread object does not have an associated thread (and is safe to destroy) after
 - it was default-constructed
 - it was moved from
 - `join()` has been called
 - `detach()` has been called
- `std::thread::operator=`
`thread& operator=(thread&& other) noexcept;`
 - ▷ If `*this` still has an associated running thread (that is, `joinable() == true`), call `std::terminate()`
 - ▷ Otherwise, assigns the state of `other` to `*this` and sets `other` to a default constructed state
 - ▷ After this call, `this->get_id()` is equal to the value of `other.get_id()` prior to the call, and `other` no longer represents a thread of execution



std::thread: Observers and Operations

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread
std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Member Functions:

- Observers:

- ▷ **joinable**: checks whether the thread running in parallel context is joinable. Typically:
 - **joinable** is **false** before it starts execution or after it has joined
 - **joinable** is **true** while excuting
 - ▷ **get_id**: returns the id of the thread
 - ▷ **native_handle** returns the underlying implementation-defined thread handle
 - ▷ **hardware_concurrency** [static]: returns the number of concurrent threads supported by the implementation

- Operations:

- ▷ **join**: waits for the thread to finish its execution
 - ▷ **detach**: permits the thread to execute independently from the thread handle
 - ▷ **swap**: swaps two thread objects



std::thread: Example

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
#include <iostream>
#include <utility> // std::ref, std::move
#include <thread> // std::this_thread::sleep_for
#include <chrono> // for sleep time: std::chrono::milliseconds(10)
void f1(int n) { for(int i=0; i<5; ++i) { std::cout << "Thread 1 executing\n";
    ++n; std::this_thread::sleep_for(std::chrono::milliseconds(10)); }
}
void f2(int& n) { for(int i=0; i<5; ++i) { std::cout << "Thread 2 executing\n";
    ++n; std::this_thread::sleep_for(std::chrono::milliseconds(10)); }
}
class foo { public: int n = 0;
    void bar() { for(int i=0; i<5; ++i) { std::cout << "Thread 3 executing\n";
        ++n; std::this_thread::sleep_for(std::chrono::milliseconds(10)); }
    }
};
class baz { public: int n = 0;
    void operator()() { for(int i=0; i<5; ++i) { std::cout << "Thread 4 executing\n";
        ++n; std::this_thread::sleep_for(std::chrono::milliseconds(10)); }
    }
};
```

// this_thread::sleep_for: Blocks the execution of the thread for sleep_duration or more



std::thread: Example

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread

std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

```

using namespace std;
int main() { int n = 0; foo f; baz b;
    // t1 is not a thread
    thread t1; cout << "t1 = " << t1.get_id() << '\n';
    // pass by value
    thread t2(f1, n + 1); cout << "t2 = " << t2.get_id() << '\n';
    // pass by reference
    thread t3(f2, ref(n)); cout << "t3 = " << t3.get_id() << '\n';
    // t4 is now running f2(). t3 is no longer a thread
    thread t4(move(t3)); cout << "t4 = " << t4.get_id() << '\n';
    // t5 runs foo::bar() on object f
    thread t5(&foo::bar, &f); cout << "t5 = " << t5.get_id() << '\n';
    // t6 runs baz::operator() on a copy of object b
    thread t6(b); cout << "t6 = " << t6.get_id() << '\n';
    t2.join(); t4.join(); t5.join(); t6.join();
    cout << "Final n = " << n << '\n';
    cout << "Final f.n (foo::n) = " << f.n << '\n';
    cout << "Final b.n (baz::n) = " << b.n << '\n';
}

```

t1 = thread::id of a non-executing thread

t2 = 140285525227264

t3 = 140285516834560

t4 = 140285516834560 // same as t3

t5 = 140285508441856

t6 = 140285500049152

Thread 4 executing

Thread 3 executing

Thread 2 executing

Thread 1 executing

Thread 4 executing

Thread 3 executing

Thread 2 executing

Thread 1 executing

Thread 4 executing

Thread 2 executing

Thread 3 executing

Thread 1 executing

Thread 4 executing

Thread 2 executing

Thread 1 executing

Thread 3 executing

Thread 4 executing

Thread 2 executing

Thread 1 executing

Thread 3 executing

Final n = 5

Final f.n (foo::n) = 5

Final b.n (baz::n) = 0



thread Programming in C++: `std::bind`

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`

`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

thread Programming in C++: `std::bind`

Sources:

- [C++11 - the new ISO C++ standard: `std::function` and `std::bind`](#), Stroustrup, 2016
- [std::bind](#), cppreference
- [std::bind](#), cplusplus
- [std::function and std::bind: what are they, and when should they be used?](#), stackoverflow
- [std::bind – Tutorial and Examples](#), thispointer
- [Bind function and placeholders in C++](#), geeksforgeeks



std::bind

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

std::thread
std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex
Solution by Atomic

Module Summary

```
template<class F, class... Args>           (since C++11)
/*unspecified*/ bind(F&& f, Args&&... args); (until C++20)
template<class F, class... Args>           (since C++20)
constexpr /*unspecified*/ bind(F&& f, Args&&... args);
```

```
template<class R, class F, class... Args>   (since C++11)
/*unspecified*/ bind(F&& f, Args&&... args); (until C++20)
template<class R, class F, class... Args>   (since C++20)
constexpr /*unspecified*/ bind(F&& f, Args&&... args);
```

- Defined in `<functional>`
- The function template `bind` generates a forwarding call wrapper for `f`. Calling this wrapper is equivalent to invoking `f` with some of its arguments bound to `args`. Parameters are:
 - `f`: Callable object (function object, pointer to function, reference to function, pointer to member function, or pointer to data member) that will be bound to some arguments
 - `args`: list of arguments to bind, with the unbound arguments replaced by the placeholders `_1`, `_2`, `_3`... of namespace `std::placeholders`



std::bind

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

- Suppose we have a callable object `f` with 3 parameters: `f(a, b, c);`
- We want a new function object `g` with only 2 parameters: `g(a, b) = f(a, 4, b);`
- Using `std::bind` to set: `auto g = std::bind(f, _1, 4, _2);` where `_1` (or `_2`) refers to the first (or second) param in `g`. `g` becomes a partial function of `f` with the middle param preset
- `std::bind` is useful in various contexts including:
 - Partial functions (or currying)
 - Reordering parameters / Defining default parameters
 - Generalized function pointer for callback and Passing functors to STL algorithms

```
#include <iostream>
#include <functional>
class MyClass { typedef std::function<void (float result)> TCallback; // shorthand to avoid long typing
    void longRunningFunction(TCallback callback) { double result = 2.7; // this function takes long time
        // do some long running task ...
        callback(result); // callback to return result
    }
    void callback(float result) { std::cout << result; } // called by longRunningFunction after its done
public: void longRunningFunctionAsync() { auto callback = // create callback as a safe function pointer
    std::bind(&MyClass::callback, this, std::placeholders::_1); // Mem fn, object, future param
    longRunningFunction(callback); // normally starts on separate thread, simple call for demo
}
};
int main() { MyClass().longRunningFunctionAsync(); } // 2.7
Programming in Modern C++
```



std::bind: Example

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
#include <iostream>
#include <utility>    // ref, move, mem_fn
#include <memory>     // make_shared, make_unique
#include <functional> // std::placeholders

void f(int n1, int n2, int n3, const int& n4, int n5) {
    std::cout << n1 << ' ' << n2 << ' ' << n3 << ' ' << n4 << ' ' << n5 << '\n';
}

int g(int n1) { return n1; }

struct Foo { int data = 10; void print_sum(int n1, int n2) { std::cout << n1+n2 << '\n'; } };

int main() { using namespace std::placeholders; // for _1, _2, _3...
    std::cout << "1) argument reordering and pass-by-reference: "; int n = 7;
    // _1 and _2 are from std::placeholders, and represent future arguments that will be passed to f1
    auto f1 = std::bind(f, _2, 42, _1, std::cref(n), n); n = 10;
    f1(1, 2, 1001); // 1 is bound by _1, 2 is bound by _2, 1001 is unused: call to f(2, 42, 1, n, 7)
    // 2 42 1 10 7

    std::cout << "2) achieving the same effect using a lambda: "; n = 7;
    auto lambda = [&nref=std::cref(n), n=n](auto a, auto b, auto /*unused*/) { f(b, 42, a, nref, n); };
    n = 10; lambda(1, 2, 1001); // same as a call to f1(1, 2, 1001)
    // 2 42 1 10 7

    std::cout << "3) nested bind subexpressions share the placeholders: ";
    auto f2 = std::bind(f, _3, std::bind(g, _3), _3, 4, 5);
    f2(10, 11, 12); // makes a call to f(12, g(12), 12, 4, 5);
    // 12 12 12 4 5
```



std::bind: Example

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
// int g(int n1) { return n1; }  
// struct Foo { int data = 10; void print_sum(int n1, int n2) { std::cout << n1+n2 << '\n'; } };  
// int main() { using namespace std::placeholders; // for _1, _2, _3...  
    std::cout << "4) bind to a pointer to member function: ";  
    Foo foo; auto f3 = std::bind(&Foo::print_sum, &foo, 95, _1);  
    f3(5); // 100  
  
    std::cout << "5) bind to a mem_fn that is a pointer to member function: ";  
    auto ptr_to_print_sum = std::mem_fn(&Foo::print_sum); // std::mem_fn generates wrapper objects  
    auto f4 = std::bind(ptr_to_print_sum, &foo, 95, _1); // for pointers to members  
    f4(5); // 100  
  
    std::cout << "6) bind to a pointer to data member: ";  
    auto f5 = std::bind(&Foo::data, _1);  
    std::cout << f5(foo) << '\n'; // 10  
  
    std::cout << "7) bind to a mem_fn that is a pointer to data member: ";  
    auto ptr_to_data = std::mem_fn(&Foo::data);  
    auto f6 = std::bind(ptr_to_data, _1);  
    std::cout << f6(foo) << '\n'; // 10  
  
    std::cout << "8) use smart pointers to call members of the referenced objects: ";  
    std::cout << f6(std::make_shared<Foo>(foo)) << ' ' << f6(std::make_unique<Foo>(foo)) << '\n';  
    // 10 10  
}
```



Race Condition & Data Race

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

Race Condition & Data Race

Sources:

- [Concurrency in C++11](#), University of Chicago
- [Race Conditions versus Data Races](#), modernescpp, 2017
- [Malicious Race Conditions and Data Races](#), modernescpp, 2017
- [C++11 Multithreading – Part 4: Data Sharing and Race Conditions](#)
- [C++11 Multithreading – Part 5: Using mutex to fix Race Conditions](#)



Race Condition & Data Race

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

- We often talk about bugs in multi-threading:
 - *Race Condition*
 - *Data Race*
- Are they same?
 - No, they are not
 - They are not a subset of one another
 - They are also neither the necessary, nor the sufficient condition for one another
- *Race Condition*: A race condition is a semantic error
 - A race condition is a situation, in which the result of an operation depends on the interleaving of certain individual operations
 - Many race conditions can be caused by data races, but this is not necessary
- *Data Race*: A data race occurs when 2 instructions from different threads access the same memory location without synchronization
 - A data race is a situation, in which at least two threads access a shared variable at the same time. At least one thread tries to modify the variable.
 - The discovery of data race can be automated
- We take examples to illustrate both



Race Condition & Data Race: Race Condition Example

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`
`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

Race Condition & Data Race: Race Condition Example



Example 1: Race Condition

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread
std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Let us write a simple program to compute:

$$\sum_{i=1}^{20} i^2 = \frac{20 \times (20 + 1) \times (2 \times 20 + 1)}{6} = 2870$$

```
#include <iostream>
using namespace std;

int accum = 0; // init accumulator
void square(int x) { accum += x * x; } // compute and accumulate product
int main() {
    for (int i = 1; i <= 20; i++) { square(i); }
    cout << " accum = " << accum << endl; // print the result
}
```

- Assuming that `x*x` is a heavy computation (fake it!) let us write a simple multi-threaded program for the above:
 - Spawn 20 threads
 - Each thread computes `square` for a distinct value
 - The accumulated result is available after the threads join



Example 1: Race Condition

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++
std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread
using namespace std;

int accum = 0; // init accumulator
void square(int x) { // called in different threads - one each for 1 .. 20
    accum += x * x; // compute and accumulate product
}
int main() {
    vector<thread> ths; // vector of threads
    for (int i = 1; i <= 20; i++) {
        ths.push_back(thread(&square, i)); // 20 threads spawned
    }
    for (auto& th : ths) {
        th.join(); // join 20 threads
    }
    cout << " accum = " << accum << endl; // print the result
}
```



Example 1: Race Condition: Random Delay

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- As we execute the multi-threaded program, it seems to correctly give the result 2870
- Does it? Always? Can we be sure?
- We need to validate our assumption that `x*x` is indeed a heavy computation, and on different threads it may take different quanta of time
- To increase the heaviness of computation of `x*x`, we insert a delay between computation of `x*x` and its accumulation
- To simulate varying situations between threads, we randomize the delay

```
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product

    // random number between 0 and 100 where std::rand() is from <cstdlib>
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); //

    // random delay: 0ms .. 100ms where std::milliseconds() is from <chrono>
    std::this_thread::sleep_for(std::chrono::milliseconds(delay));

    accum += p; // accumulate product
}
```

- We try again!



Example 1: Race Condition: Random Delay

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;

int accum = 0; // init accumulator
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}

int main() {
    vector<thread> ths; // vector of threads
    for (int i = 1; i <= 20; i++) {
        ths.push_back(thread(&square, i)); // 20 threads spawned
    }
    for (auto& th : ths) {
        th.join(); // join 20 threads
    }
    cout << " accum = " << accum << endl; // print the result
}
```



Example 1: Race Condition: Random Delay + Repeat

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread

std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- As we execute the modified multi-threaded program with delays, it seems still to correctly give the result 2870
- We keep trying. Running it over and over again to be convinced of the correctness (someone told that thread programming is tricky)
- When we are almost certain of the correctness, **suddenly on the 37th run, we get 2845!**
- Was it a computer error, false observation? We try another 100+ times and always get 2870!
- We decide we need to automate the runs:
 - We run (trial) in a in an infinite loop
 - We break the loop if the trial fails to produce correct result

```
int main() {  
    int trial_count = 0; // counting trials before failure  
    do {  
        ++trial_count; // increment trial counter  
        accum = 0; // reset to start a trial  
        // codes for vector of threads, 20 threads spawned, join 20 threads  
    } while (accum == 2870); // 1^2+2^2+...20^2 = 2870: infinite loop!!!  
}
```

- Correct program will loop forever! But Murphy says: *If anything can go wrong, it will*



Example 1: Race Condition: Random Delay + Repeat

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;

int accum = 0; // init accumulator
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}

int main() { int trial_count = 0; // counting trials before failure
    do { ++trial_count; // increment trial counter
        if (0 == trial_count % 100) // message after every 100 trials - that the process is alive
            cout << "trials = " << trial_count << endl;
        accum = 0; // reset to start a trial
        vector<thread> ths; // vector of threads
        for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
        for (auto& th : ths) { th.join(); } // join 20 threads
    } while (accum == 2870); // 1^2+2^2+...20^2 = 2870: infinite loop!!!
    cout << "trials = " << trial_count << " accum = " << accum << endl; // print if there is bad result
}
```

Programming in Modern C++



Example 1: Race Condition: Random Delay + Repeat: Results

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

`std::thread`

`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- Murphy is correct!¹. Every time we run, the loop breaks after some trials (not very large in fact)
- Here are the first 20 runs (of multiple trials). Every time there is some trial which **falls short of 2870**

```
trials = 56 accum = 2845
trials = 1 accum = 2806
trials = 221 accum = 2470
trials = 825 accum = 2806
trials = 1502 accum = 2861
trials = 487 accum = 2861
trials = 113 accum = 2470
trials = 156 accum = 2861
trials = 1120 accum = 2581
trials = 914 accum = 2645
trials = 1279 accum = 2726
trials = 932 accum = 2806
trials = 1120 accum = 2581
trials = 174 accum = 2845
trials = 190 accum = 2645
trials = 229 accum = 2546
trials = 802 accum = 2821
trials = 67 accum = 2614
trials = 784 accum = 2869
trials = 295 accum = 2854
```

¹ *which is a logical contradiction by Murphy's law*



Example 1: Race Condition: Analysis

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread
std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

- So what is going wrong? We have hit a *Race Condition*
- When the compiler processes `accum += x * x;`, reading the current value of `accum` and setting the updated value is not an *atomic* (meaning *indivisible*) event. Let us re-write `square` to capture this:

```
int t1 = x * x;
int t2 = accum;
accum = t2 + t1;
```

- Now, let us assume that we working with only 2 threads (for 1 & 2). The threads are interleaved over time and a possible sequence is:

<code>// Thread 1 (Th1)</code>	<code>// Thread 2 (Th2)</code>	
<code>int t1 = 1 * 1;</code>	<code>int t1 = 2 * 2;</code>	<code>// Th2.t1 = 4</code>
	<code>t2 = accum;</code>	<code>// Th1.t1 = 1</code>
<code>t2 = accum;</code>		<code>// Th2.t2 = 0</code>
<code>accum = t2 + t1;</code>		<code>// Th1.t2 = 0</code>
	<code>accum = t2 + t1;</code>	<code>// accum = 1</code>
		<code>// accum = 4</code>

- We end up with `accum` as 4, instead of the correct 5
- It also makes clear why a wrong result will always be less
- Let us now provide two solutions to the race condition problem using
 - Mutex
 - Atomic



Race Condition & Data Race: Race Condition Example: Solution by Mutex

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

`std::thread`
`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

Race Condition & Data Race: Race Condition Example: Solution by Mutex



Example 1: Race Condition: Solution by Mutex

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

- A mutex (mutual exclusion) allows us to encapsulate blocks of code that should only be executed in one thread at a time. Keeping the main function the same:

```
int accum = 0;
mutex accum_mutex; // mutex variable

void square(int x) {
    int temp = x * x;
    accum_mutex.lock(); // gets the lock on accum_mutex
    accum += temp;
    accum_mutex.unlock(); // release the lock on accum_mutex
}
```

- We try running the program repeatedly again and the problem should now be fixed
- The first thread that calls `lock()` gets the lock
- During this time, all other threads that call `lock()`, will wait at that line for the mutex to be unlocked. Creates a *Critical Section*
- It is important to introduce the variable `temp`, since we want the `x * x` calculations to be outside the lock-unlock block, otherwise we would be hogging the lock while we are running our heavy calculations



Example 1: Race Condition: Solution by Mutex

Module M58

Partha Pratim Das

Objectives & Outlines

thread Programming in C++

std::thread

std::bind

Race Condition & Data Race

Race Condition Example

Solution by Mutex

Solution by Atomic

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <mutex> // mutex
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
int accum = 0; // init accumulator
mutex accum_mutex; // mutex variable
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum_mutex.lock(); // gets the lock on accum_mutex
    accum += p; // accumulate product
    accum_mutex.unlock(); // release the lock on accum_mutex
}
int main() { int trial_count = 0; // counting trials before failure
    do { ++trial_count; // increment trial counter
        accum = 0; // reset to start a trial
        vector<thread> ths; // vector of threads
        for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
        for (auto& th : ths) { th.join(); } // join 20 threads
    } while (accum == 2870); // 1^2+2^2+...20^2 = 2870: infinite loop!!!
    cout << "trials = " << trial_count << " accum = " << accum << endl; // print if there is bad result
}
```

Programming in Modern C++



Race Condition & Data Race: Race Condition Example: Solution by Atomic

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

`std::thread`
`std::bind`

Race Condition & Data Race

Race Condition Example

Solution by Mutex
Solution by Atomic

Module Summary

Race Condition & Data Race: Race Condition Example: Solution by Atomic



Example 1: Race Condition: Solution by Atomic

Module M58

Partha Pratim Das

Objectives & Outlines

thread
Programming in C++

std::thread
std::bind

Race Condition & Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

- With Mutex the problem gets fixed. The program does not produce a wrong result even after 6000+ trials
- Interestingly, C++11 offers even nicer abstractions to solve this problem. For instance, the `atomic` container:

```
#include <atomic>
```

```
atomic<int> accum(0); // makes accum and initializes to 0
```

```
void square(int x) {  
    accum += x * x;  
}
```

- We do not need to introduce temp here, since `x * x` will be evaluated before handed off to `accum`, so it will be outside the atomic event
- However, we will continue to show the solution using the temporary



Example 1: Race Condition: Solution by Atomic

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

std::thread
std::bind

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex
Solution by Atomic

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <atomic> // atomic
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
atomic<int> accum(0); // makes accum and initializes to 0
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}
int main() { int trial_count = 0; // counting trials before failure
    do { ++trial_count; // increment trial counter
        accum = 0; // reset to start a trial
        vector<thread> ths; // vector of threads
        for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
        for (auto& th : ths) { th.join(); } // join 20 threads
    } while (accum == 2870); // 1^2+2^2+...20^2 = 2870: infinite loop!!!
    cout << "trials = " << trial_count << " accum = " << accum << endl; // print if there is bad result
}
```

- Works fine. Does not produce a wrong result even after 5000+ trials



Module Summary

Module M58

Partha Pratim
Das

Objectives &
Outlines

thread
Programming in
C++

`std::thread`

`std::bind`

Race Condition &
Data Race

Race Condition
Example

Solution by Mutex

Solution by Atomic

Module Summary

- Introduced the notion of concurrent programming in C++11 using thread support
- Explored library support through `std::thread` and `std::bind`
- Exposed to the bugs in thread programming - race condition and data race
- Discussed examples of thread programs with bugs and their solution



Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Programming in Modern C++

Module M59: C++11 and beyond: Concurrency: Part 2

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- Introduced the notion of concurrent programming in C++11 using thread support
- Explored library support through `std::thread` and `std::bind`
- Exposed to the bugs in thread programming - race condition and data race
- Discussed examples of thread programs with bugs and their solution



Module Objectives

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- To understand synchronization issues in multi-thread programming in C++
- To study various synchronization mechanisms through example
- To self-study the details of synchronization mechanisms:
 - *Mutex*
 - *Lock*
 - *Atomics*
 - *Condition Variable*
 - *Future and Promises*
 - *Async*



Module Outline

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- 1 Threads
- 2 Race Condition and Data Race
 - Solution by Mutex
 - Solution by Lock
 - Solution by Atomic
 - Solution by Future
 - Solution by Async
- 3 Synchronization
 - Thread Local
- 4 Self-Study
 - Mutual Exclusion
 - Locks
 - Deadlock
 - Atomics
 - Sync: Condition Variables
 - Sync: Futures and Promises
 - Async
 - Practice Examples
- 5 Module Summary

Programming in Modern C++

Partha Pratim Das

M59.4



Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Sources:

- [C++11 - the new ISO C++ standard: Threads](#), Stroustrup, 2016
- [C++11 Standard Library Extensions — Concurrency: Threads](#), isocpp
- [std::thread](#), cppreference
- [Concurrency memory model](#), isocpp.org
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses
- [A tutorial on modern multithreading and concurrency in C++](#), 2020
- **C++11 Multi-threading Tutorials**: Parts 1-8, [thisPointer](#)
 - [C++11 Multithreading – Part 1 : Three Different ways to Create Threads](#)
- **C++20 Concurrency**: Parts 1-3, [Gajendra Gulgulia](#), 2021
 - [C++20 Concurrency: Part 1: synchronized output stream](#)
 - [C++20 Concurrency: Part 2: jthreads](#)
 - [C++20 Concurrency: Part 3: request_stop and stop_token for std::jthread](#)

Threads



Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- A **thread** is a *light-weight process*
- We have learnt the basic thread operations using `std::thread`:
 - Create a thread
 - Pass parameter/s to a thread - directly or by `std::bind`
 - Return result/s from a thread - directly or by `std::bind`
 - Join threads
- We have also observed race condition and data race in simple multi-threaded program
- To alleviate such bugs we need to understand the synchronization of threads and various mechanisms for it



Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race



Race Condition and Data Race

(Module 58)

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- We often talk about bugs in multi-threading:
 - *Race Condition*
 - *Data Race*
- Are they same?
 - No, they are not
 - They are not a subset of one another
 - They are also neither the necessary, nor the sufficient condition for one another
- *Race Condition*: A race condition is a semantic error
 - A race condition is a situation, in which the result of an operation depends on the interleaving of certain individual operations
 - Many race conditions can be caused by data races, but this is not necessary
- *Data Race*: A data race occurs when 2 instructions from different threads access the same memory location without synchronization
 - A data race is a situation, in which at least two threads access a shared variable at the same time. At least one thread tries to modify the variable.
 - The discovery of data race can be automated
- We take examples to illustrate both



Example 1

(Module 58)

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- Let us write a simple program to compute:

$$\sum_{i=1}^{20} i^2 = \frac{20 \times (20 + 1) \times (2 \times 20 + 1)}{6} = 2870$$

- Assuming that `x*x` is a heavy computation (fake it!) we developed a simple multi-threaded program for the above:
 - Spawn 20 threads
 - Each thread computes `square` for a distinct value
 - The accumulated result is available after the threads join
- We added random delay and repeated run support to setup scenarios for race conditions to be observed. We observed that bugs exist
- We have also discussed two fixes – by `mutex` and by `atomic` which we will recap here
- We also discuss other solutions by `lock`, `future`, and `async`



Example 1: Random Delay + Repeat

(Module 58)

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;

int accum = 0; // init accumulator
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}

int main() { int trial_count = 0; // counting trials before failure
    do { ++trial_count; // increment trial counter
        if (0 == trial_count % 100) // message after every 100 trials - that the process is alive
            cout << "trials = " << trial_count << endl;
        accum = 0; // reset to start a trial
        vector<thread> ths; // vector of threads
        for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
        for (auto& th : ths) { th.join(); } // join 20 threads
    } while (accum == 2870); // 1^2+2^2+...20^2 = 2870: infinite loop!!!
    cout << "trials = " << trial_count << " accum = " << accum << endl; // print if there is bad result
}
```

Programming in Modern C++



Example 1: Solution by Mutex

(Module 58)

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Example 1: Solution by Mutex



Example 1: Solution by Mutex

(Module 58)

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- A mutex (mutual exclusion) allows us to encapsulate blocks of code that should only be executed in one thread at a time. Keeping the main function the same:

```
int accum = 0;
mutex accum_mutex; // mutex variable

void square(int x) {
    int temp = x * x;
    accum_mutex.lock(); // gets the lock on accum_mutex
    accum += temp;
    accum_mutex.unlock(); // release the lock on accum_mutex
}
```

- We try running the program repeatedly again and the problem should now be fixed
- The first thread that calls `lock()` gets the lock
- During this time, all other threads that call `lock()`, will wait at that line for the mutex to be unlocked. Creates a *Critical Section*
- It is important to introduce the variable `temp`, since we want the `x * x` calculations to be outside the lock-unlock block, otherwise we would be hogging the lock while we are running our heavy calculations



Example 1: Solution by Mutex

(Module 58)

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <mutex> // mutex
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
int accum = 0; // init accumulator
mutex accum_mutex; // mutex variable
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum_mutex.lock(); // gets the lock on accum_mutex
    accum += p; // accumulate product
    accum_mutex.unlock(); // release the lock on accum_mutex
}
int main() {
    vector<thread> ths; // vector of threads
    for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
    for (auto& th : ths) { th.join(); } // join 20 threads
    cout << " accum = " << accum << endl; // print final value
}
```



Example 1: Solution by Lock

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Example 1: Solution by Lock



Example 1: Solution by Lock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- A lock is an object that can hold a reference to a mutex and may `unlock()` the mutex during the lock's destruction (such as when leaving block scope)

```
int accum = 0;
mutex accum_mutex; // mutex variable

void square(int x) {
    int temp = x * x;
    std::unique_lock<std::mutex> // acquires and owns the lock on accum_mutex
        lck(accum_mutex);
    //accum_mutex.lock(); // gets the lock on accum_mutex
    accum += temp;
    //accum_mutex.unlock(); // release the lock on accum_mutex
} // release the lock and ownership on accum_mutex
```

- Use of `lock` makes the coding and understanding simpler than using bare `mutex`
- `std::unique_lock` has the similar resource ownership advantages as of `std::unique_ptr`
- Particularly useful when we have multiple resources to mutually exclusively manage



Example 1: Solution by Lock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <mutex> // mutex, unique_lock
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
int accum = 0; // init accumulator
mutex accum_mutex; // mutex variable
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    std::unique_lock<std::mutex> lck(accum_mutex); // acquires and owns the lock on accum_mutex
    accum += p; // accumulate product
}
int main() {
    vector<thread> ths; // vector of threads
    for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
    for (auto& th : ths) { th.join(); } // join 20 threads
    cout << " accum = " << accum << endl; // print final value
}
```



Example 1: Solution by Atomic

(Module 58)

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Example 1: Solution by Atomic



Example 1: Solution by Atomic

(Module 58)

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- With Mutex / Lock the problem gets fixed. The program does not produce a wrong result even after 6000+ trials
- Interestingly, **C++11** offers even nicer abstractions to solve this problem. For instance, the **atomic** container:

```
#include <atomic>
```

```
atomic<int> accum(0); // makes accum and initializes to 0
```

```
void square(int x) {  
    accum += x * x;  
}
```

- We do not need to introduce temp here, since **x * x** will be evaluated before handed off to **accum**, so it will be outside the atomic event
- However, we will continue to show the solution using the temporary



Example 1: Solution by Atomic

(Module 58)

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <atomic> // atomic
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
atomic<int> accum(0); // makes accum and initializes to 0
void square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}
int main() {
    vector<thread> ths; // vector of threads
    for (int i = 1; i <= 20; i++) { ths.push_back(thread(&square, i)); } // 20 threads spawned
    for (auto& th : ths) { th.join(); } // join 20 threads
    cout << " accum = " << accum << endl; // print final value
}
```

- Works fine. Does not produce a wrong result even after 5000+ trials



Example 1: Solution by Future

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Example 1: Solution by Future



Example 1: Solution by Future

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- Future, represented by `std::future`, is a way to access the results of asynchronous operations
- Imagine if our main `thread A` wants to open a new `thread B` to perform some of our expected tasks and return me a result. At this time, `thread A` may be busy with other things and have no time to take into account the results of `thread B`. So we naturally hope to get the result of `thread B` at a certain time
- Before the introduction of `std::future` in **C++11**, the usual practice used to be:
 - Create a `thread A`
 - start `task B` in `thread A`
 - send an `event` when it is ready, and
 - save the result in a *global variable*
 - The main function thread A is doing other things. When the result is needed, a thread is called to `wait` for the function to get the result of the execution
- The `std::future` provided by **C++11** simplifies this process and can be used to get the results of asynchronous tasks. Naturally, we can easily imagine it as a simple means of thread synchronization, namely the *barrier*
- We engage Example 1 to illustrate the way Future works



Example 1: Solution by Future

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <future> // future
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
int accum = 0; // define accumulator
void square(future<int>& fut) { // called in different threads - one each for 1 .. 20
    int x = fut.get(); // get parameter from future
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    accum += p; // accumulate product
}
int main() {
    vector<promise<int>> vp; /*promises*/ vector<future<int>> vf; /*futures*/ vector<thread> vt;
    for (int i = 0; i < 20; i++) {
        vp.push_back(promise<int>()); // vector of promise objects
        vf.push_back(vp[i].get_future()); // vector of engaged future objects from promise objects
        vt.push_back(thread(&square, ref(vf[i]))); // vector of threads - pass future object
        vp[i].set_value(i+1); // fulfil promise with needed value
    }
    for (auto& t : vt) { t.join(); } // join 20 threads
    cout << " accum = " << accum << endl; // print final value
}
```



Example 1: Solution by Async

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Example 1: Solution by Async



Example 1: Solution by Async

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- An even higher level of abstraction that avoids the use of promise and future directly, talking in terms of *tasks* is *async* given in `std::future`. Consider the following example:

```
#include <iostream>
#include <future> // future
using namespace std;

int square(int x) { return x * x; }
int main() {
    auto a = async(&square, 10); // returns a future<int>
    int v = a.get();             // waits to fulfil the promise

    cout << "The thread returned " << v << endl;
}
```

- The *async* construct uses an object pair called a *promise* and a *future*
- The former has made a promise to eventually provide a value
- The *future* is linked to the *promise* and can at any time try to retrieve the value by `get()`
- If the *promise* has not been fulfilled yet, it will simply wait until the value is ready
- The *async* hides most of this for us, except that it returns in this case a `future<int>` object



Example 1: Solution by Async

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>
#include <vector>
#include <thread> // thread, this_thread::sleep_for
#include <future> // future
#include <chrono> // chrono::milliseconds
#include <cstdlib> // rand()
using namespace std;
int square(int x) { // called in different threads - one each for 1 .. 20
    int p = x * x; // compute product
    int delay = (int)((double)std::rand() / (double)(RAND_MAX)* 100); // random number between 0 and 100
    std::this_thread::sleep_for(std::chrono::milliseconds(delay)); // random delay: 0ms .. 100ms
    return p;
}
int main() {
    int accum = 0; // define accumulator
    vector<future<int>> fts; // vector of future objects
    for (int i = 1; i <= 20; i++) { fts.push_back(async(&square, i)); } // 20 future objects
    for (auto& ft : fts) { accum += ft.get(); } // wait to get value from future and accumulate
    cout << " accum = " << accum << endl; // print final value
}
```

- Works fine. Does not produce a wrong result even after 30000+ trials



Synchronization

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization



Synchronization Errors: Symptoms and Causes

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- **Conflicting access to shared memory**

- one thread begins an operation on shared memory, is suspended, and leaves that memory region incompletely transformed
- a second thread is activated and accesses the shared memory in the corrupted state, causing errors in its operation and potentially errors in the operation of the suspended thread when it resumes

- **Race Conditions**

- correct operation depends on the order of completion of two or more independent activities
- the order of completion is not deterministic

- **Deadlock**

- two or more tasks each own resources needed by the other preventing either one from running so neither ever completes and never releases its resource

- **Starvation**

- a high priority thread dominates CPU resources, preventing lower priority threads from running often enough or at all

- **Priority inversion**

- a low priority task holds a resource needed by a higher priority task, blocking it from running



Synchronization

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- A program may need multiple threads to share some data
- If access is not controlled to be sequential, then shared data may become corrupted
 - One thread accesses the data, begins to modify the data, and then is put to sleep because its time slice has expired. The problem arises when the data is in an incomplete state of modification.
 - Another thread awakes and accesses the data, that is only partially modified. The result is very likely to be corrupt data.
- The process of making access serial is called serialization or synchronization
- Synchronization may be achieved in various ways including:
 - *Mutex* (self-study)
 - *Lock* (self-study)
 - *Atomics* (self-study)
 - *Condition Variable* (self-study)
 - *Future and Promises* (self-study)
 - *Async* (self-study)



Synchronization: `thread_local`

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: `thread_local`

Sources:

- [Thread-local storage](#), [isocpp.org](#)
- [Storage class specifiers](#), [cppreference](#)
- [Thread-Local Data](#), [modernescpp](#), 2016
- [What does the `thread\_local` mean in C++11?](#), [stackoverflow](#)



thread_local

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- `thread_local` is a storage class specifier. Thread local data will be created for each thread as needed
- `thread_local` data exclusively belongs to the thread and behaves like `static` data
- Created at its first use and *lifetime* bound to the *lifetime of the thread* (*lifetime* in **Module 13, 23, & 35**)

Thread Local

```
#include <iostream>
#include <thread>
#include <vector>
using namespace std;
thread_local int i = 0; //

void f(int newval) { i = newval; }
void g() { cout << i; }
void threadfunc(int id) {
    f(id); ++i; /*          */ g();
}
int main() { i = 9;
    vector<thread> th;
    for(int i = 1; i < 4; ++i)
        th.push_back(thread(threadfunc, i));
    for(auto& t: th) t.join();
    cout << i << endl;
}
// 2349, 3249, 4239, 4329, 2439 or 3429
```

Global

```
#include <iostream>
#include <thread>
#include <vector>
using namespace std;
// int i = 0;

void f(int newval) { i = newval; }
void g() { cout << i; }
void threadfunc(int id) {
    f(id); ++i; /*          */ g();
}
int main() { i = 9;
    vector<thread> th;
    for(int i = 1; i < 4; ++i)
        th.push_back(thread(threadfunc, i));
    for(auto& t: th) t.join();
    cout << i << endl;
}
```



Self-Study

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Self-Study



Synchronization: mutex

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: mutex

Sources:

- [Mutual exclusion](#), isocpp.org
- [std::mutex](#), cplusplus
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



mutex

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- We have used mutex from `<mutex>` in the solution for Example 1
- A mutex is a primitive object used for controlling access in a multi-threaded system

```
std::mutex m;  
int sh; // shared data  
// ...  
m.lock();  
// manipulate shared data  
sh+=1;  
m.unlock();
```

- Only one thread at a time can be in the region of code between the `lock()` and the `unlock()` (**critical region**)
- If a second thread tries `m.lock()` while a first thread is executing in that region, that second thread is blocked until the first executes the `m.unlock()`
- There may give rise to serious problems like:
 - What if a thread “forgets” to `unlock()`?
 - What if a thread tries to `lock()` the same mutex twice?
 - What if a thread waits a very long time before doing an `unlock()`?
 - What if a thread needs to `lock()` two mutexes to do its job?
 - What if ...?



mutex

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- In addition to `lock()`, a mutex has a `try_lock()` operation which can be used to try to get into the critical region without the risk of getting blocked:

```
std::mutex m;  
int sh; // shared data  
// ...  
if (m.try_lock()) { // either get the lock or fail and continue - no blocking  
    // manipulate shared data  
    sh+=1;  
    m.unlock();  
}  
else { /* maybe do something else */ }
```

- Use a `recursive_mutex` to acquire it more than once by a thread in a recursive or co-recursive function
- We can also set a duration (relative time) to try for a lock:
`m.try_lock_for(std::chrono::seconds(10)) // get it in the next 10 seconds or fail`
- Or we may want to wait until a fixed point in time, a `time_point`:
`m.try_lock_until(midnight) // wait till midnight or fail`
- A `recursive_timed_mutex` is a `recursive_mutex` that can be timed



Synchronization: lock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: lock

Sources:

- [Locks](#), isocpp.org
- [std::lock](#), cplusplus
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



lock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- We have used lock from `<mutex>` in the solution for Example 1
- A lock is an object that can hold a reference to a mutex and may `unlock()` the mutex during the lock's destruction (such as when leaving block scope)
- A thread may use a lock to aid in managing mutex ownership in an exception safe manner
- That is, a lock implements *Resource Acquisition Is Initialization (RAII)* for mutual exclusion (Recall RAII in smart pointers). For example:

```
std::mutex m;  
int sh; // shared data  
// ...  
void f() {  
    // ...  
    std::unique_lock<std::mutex> lck(m); // m.lock()  
    // manipulate shared data:  
    // lock will be released even if this code throws an exception  
    sh+=1;  
} // m.unlock()
```

- A lock can be *moved* (the purpose of a lock is to represent local ownership of a non-local resource), but not *copied* (which copy would own the resource/mutex?)



lock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- This straightforward picture of a lock is clouded by `unique_lock` having facilities to do just about everything a mutex can, but *safer* and *simpler*
- For example, we can use a `unique_lock` to do `try_lock`:

```
std::mutex m;
int sh; // shared data
// ...
void f() {
    // ...
    std::unique_lock<std::mutex> lck(m, std::defer_lock); // make a lock, but do not
    // ...                                           // acquire the mutex
    if (lck.try_lock()) {
        // manipulate shared data:
        sh+=1;
    }
    else { /* maybe do something else */ }
}
```

- Similarly, `unique_lock` supports `try_lock_for()` and `try_lock_until()`
- What you get from using a lock rather than the mutex directly is exception handling and protection against forgetting to `unlock()`



Synchronization: lock: Deadlock

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: lock: Deadlock



lock: Deadlock

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- What if we need two resources represented by two mutexes? The naive way is to acquire the mutexes in order:

```
std::mutex m1;
std::mutex m2;
int sh1;    // shared data
int sh2
// ...
void f() {
    // ...
    std::unique_lock<std::mutex> lck1(m1);
    std::unique_lock<std::mutex> lck2(m2);
    // manipulate shared data:
    sh1+=sh2;
}
```

- This has the potentially deadly flaw that some other thread could try to acquire `m1` and `m2` in the opposite order so that each had one of the locks needed to proceed and would wait forever for the second (*deadlock*)
- With many locks in a system, that is a real danger



lock: Deadlock

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- The standard locks provide two functions for (safely) trying to acquire two or more locks:

```
void f() { // ...
    std::unique_lock<std::mutex> lck1(m1, std::defer_lock); // make locks but
    std::unique_lock<std::mutex> lck2(m2, std::defer_lock); // do not yet try to
    std::unique_lock<std::mutex> lck3(m3, std::defer_lock); // acquire the mutexes
    std::lock(lck1, lck2, lck3);
    // manipulate shared data:
}
```

- The implementation of `lock()` is carefully crafted to avoid *deadlock*
- In essence, it will do the equivalent to careful use of `try_lock()`s
- If `lock()` fails to acquire all locks it will throw an exception
- If you prefer to use `try_lock()`s yourself, there is an equivalent to `lock()` to help:

```
void f() { int x; // ...
    std::unique_lock<std::mutex> lck1(m1, std::defer_lock); // make locks but
    std::unique_lock<std::mutex> lck2(m2, std::defer_lock); // do not yet try to
    std::unique_lock<std::mutex> lck3(m3, std::defer_lock); // acquire the mutexes
    if ((x = try_lock(lck1, lck2, lck3))!=-1) { // manipulate shared data:
    } else { // x holds the index of a mutex we could not acquire
        // for example, if lck2.try_lock() failed x==1
    }
}
```



Synchronization: atomic

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: atomic

Sources:

- [Atomics](http://isocpp.org), isocpp.org
- [std::atomic](http://en.cppreference.com/atomic), cplusplus
- [atomic Weapons: The C++ Memory Model and Modern Hardware](http://herbsutter.com), herbsutter.com, 2013
- [C++ and Beyond](http://ericniebler.com), Scott Meyers, Herb Sutter, and Andrei Alexandrescu, 2010-14
- [An Overview of the New C++ \(C++11/14\)](http://ericniebler.com), Scott Meyers Training Courses



atomic

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- We have used atomic from `<atomic>` in the solution for Example 1
- Each instantiation of the `std::atomic` template defines an atomic type. **If one thread writes to an atomic object while another thread reads from it, the behavior is well-defined**
- `std::atomic` is neither copyable nor movable
- Type aliases are provided for `bool` (`std::atomic_bool`) and integral types like `int`, `short`, etc.

```
#include <iostream>          // std::cout
#include <atomic>             // std::atomic, std::atomic_flag, ATOMIC_FLAG_INIT
#include <thread>             // std::thread, std::this_thread::yield
#include <vector>             // std::vector
std::atomic<bool> ready (false);
std::atomic_flag winner = ATOMIC_FLAG_INIT; // set false. atomic_flag has no load / store

void count1m (int id) {
    while (!ready) { std::this_thread::yield(); } // all threads wait for the ready signal to start
    for (volatile int i=0; i<1000000; ++i)        // go!, count to 1 million
        if (!winner.test_and_set()) // atomically sets the flag to true and obtains its previous value
            { std::cout << "thread #" << id << " won!\n"; }
};

int main () { std::vector<std::thread> threads;
    std::cout << "spawning 10 threads that count to 1 million...\n";
    for (int i=1; i<=10; ++i) threads.push_back(std::thread(count1m,i));
    ready = true; // signal ready to start
    for (auto& th : threads) th.join();
} // thread #8 won! // thread #4 won! // thread #1 won! // thread #9 won! ...
```




Synchronization: `condition_variable`

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: `condition_variable`

Sources:

- [Condition variables](#), [isocpp.org](#)
- [std::condition_variable](#), [cplusplus](#)
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



condition_variable

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- A condition variable is an object able to block the calling thread until notified to resume
- It uses a `unique_lock` (over a `mutex`) to lock the thread when one of its *wait functions* is called
- The thread remains blocked until woken up by another thread that calls a *notification function* on the same `condition_variable` object
- Objects of type `condition_variable` always use `unique_lock<mutex>` to wait: for an alternative that works with any kind of lockable type, use `condition_variable_any`
- Condition variables provide good solutions to deadlock problem too



condition_variable

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomsics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <iostream>           // std::cout
#include <thread>              // std::thread
#include <mutex>               // std::mutex, std::unique_lock
#include <condition_variable> // std::condition_variable

std::mutex mtx;
std::condition_variable cv;
bool ready = false;
void print_id (int id) { std::unique_lock<std::mutex> lck(mtx);
    while (!ready) cv.wait(lck);
    // ...
    std::cout << "thr. " << id << ' ';
}
void go() { std::unique_lock<std::mutex> lck(mtx);
    ready = true;
    cv.notify_all();
}
int main () {
    std::thread threads[10];
    // spawn 10 threads:
    for (int i=0; i<10; ++i) threads[i] = std::thread(print_id,i);
    std::cout << "10 threads ready to race...\n";
    go(); // go!
    for (auto& th : threads) th.join();
}
```



Synchronization: future and promise

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: future and promise

Sources:

- [Futures and promises](#), [isocpp.org](#)
- [std::future](#), [cplusplus](#)
- [std::promise](#), [cplusplus](#)
- [Copying and rethrowing exceptions](#), [isocpp.org](#)
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



future and promise

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- **C++11** offers **future** and **promise** for returning a value from a task spawned on a separate thread, and `packaged_task` to help launch tasks
- The important point about **future** and **promise** is that they enable a transfer of a value between two tasks without explicit use of a lock; *the system* implements the transfer efficiently
- The basic idea is simple: When a task wants to return a value to the thread that launched it, it puts the value into a **promise**. Somehow, the implementation makes that value appear in the **future** attached to the promise
- The caller (typically the launcher of the task) can then read the value
- The standard provides three kinds of futures, **future** for most simple uses, and **shared_future** and **atomic_future** for some trickier cases
- Here, we will just present **future** because it is the simplest and does all we need. If we have a **future<X>** called **f**, we can **get()** a value of type **X** from it:

```
X v = f.get(); // if necessary wait for the value to get computed
```
- If the value is not there yet, our thread is blocked until it arrives
- If the value could not be computed and the task will throw an exception, calling **get()** will **rethrow** that exception to the code calling **get()**



future and promise

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atoms

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- We might not want to wait for a result, so we can ask the **future** if a result has arrived:

```
if (f.wait_for(0)) { // there is a value to get()
    // do something
}
else { /* do something else */ }
```
- However, the main purpose of future is to provide that simple **get()**
- The main purpose of **promise** is to provide a simple **set()** to match **future**'s **get()**
- If you have a promise and need to send a result of type X (back) to a future, there are basically two things you can do:

```
    ○ pass a value
    ○ pass an exception
try { X res;
    // compute a value for res
    p.set_value(res);
}
catch (...) { /* could not compute res */ p.set_exception(std::current_exception()); }
if (f.wait_for(0)) { // there is a value to get()
    // do something
}
else { /* do something else */ }
```



Synchronization: `async`

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Synchronization: `async`

Sources:

- [async](#), [isocpp.org](#)
- [std::async](#), [cplusplus](#)
- [An Overview of the New C++ \(C++11/14\)](#), Scott Meyers Training Courses



async

- We have used `async` from `<future>` in the solution for Example 1
- `async` is a way for the programmer to rise above the messy threads-plus-lock level of programming:

```
#include <iostream>    // cout
#include <future>       // async
#include <vector>       // vector
#include <numeric>      // accumulate
using namespace std;

template<class T, class V> struct Accum { // simple accumulator function object
    T* b; T* e; V val; Accum(T* bb, T* ee, const V& v): b{bb}, e{ee}, val{v} { }
    V operator()() { return std::accumulate(b, e, val); }
};

double comp(vector<double>& v) { // spawn many tasks if v is large enough
    if (v.size() < 100) return std::accumulate(v.begin(), v.end(), 0.0); // to short to multi-thread
    auto f0{async(Accum<double, double>{&v[0], &v[v.size()/4], 0.0})}; // 0 125
    auto f1{async(Accum<double, double>{&v[v.size()/4], &v[v.size()/2], 0.0})}; // 125 250
    auto f2{async(Accum<double, double>{&v[v.size()/2], &v[v.size()*3/4], 0.0})}; // 250 375
    auto f3{async(Accum<double, double>{&v[v.size()*3/4], &v[v.size()], 0.0})}; // 375 500
    return f0.get()+f1.get()+f2.get()+f3.get(); // wait for the values as promised
}

int main () { vector<double> v;
    for (int i = 1; i <= 500; ++i) v.push_back(i); // fill the vector
    cout << "Sum = " << accumulate(v.begin(), v.end(), 0.0) << endl; // sequential 125250
    cout << "Sum = " << comp(v) << endl; // multi-thread 125250
}
```




Race Condition and Data Race: Example 2

Module M59

Partha Pratim Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

Race Condition and Data Race: Practice Examples



Example 2

Module M59

Partha Pratim
Das

Objectives &
Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

```
#include <functional>
#include <iostream>
#include <thread>
#include <vector>

struct Account { int balance{100}; }; // initially each account is loaded with Rs. 100
void addMoney(Account& to, int amount)
    { to.balance += amount; } // add amount to account with synchronization
int main() { Account account;

    std::vector<std::thread> vecThreads(100);
    for (auto& thr: vecThreads)
        thr = std::thread(addMoney, std::ref(account), 50); // add Rs. 50 to the account

    for (auto& thr: vecThreads) thr.join();
    std::cout << "account.balance: " << account.balance << std::endl; // final balance
}
```

- 100 threads are adding Rs. 50 to the same account using function `addMoney` but without synchronisation
- Final balance differs between Rs. 5000 and Rs. 5100 and we have a data race



Example 3

- Let us consider a function that transfers money from one account to another.
- In the single-threaded case, all is fine:

```
#include <iostream>

struct Account { int balance{100}; }; // initially each account is loaded with Rs. 100
void transferMoney(int amount, Account& from, Account& to) {
    if (from.balance >= amount) { // transfer is allowed only if there is enough fund
        from.balance -= amount;
        to.balance += amount;
    }
}

int main() { Account account1, account2; // two accounts for mutual transfers
    transferMoney(50, account1, account2); // two transfers between accounts
    transferMoney(130, account2, account1); // in sequential, total order

    std::cout << "account1.balance: " << account1.balance << std::endl;
    std::cout << "account2.balance: " << account2.balance << std::endl;
}

// account1.balance: 180 // 100 - 50 + 130
// account2.balance: 20 // 100 + 50 - 130
```



Example 3

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- Let us multi-thread the program

```
#include <iostream>
#include <functional> // ref
#include <thread>      // thread, this_thread::sleep_for
#include <chrono>      // chrono::nanoseconds
struct Account { int balance{100}; }; // initially each account is loaded with Rs. 100
void transferMoney(int amount, Account& from, Account& to) {
    if (from.balance >= amount) { // transfer is allowed only if there is enough fund
        from.balance -= amount;
        std::this_thread::sleep_for(std::chrono::nanoseconds(1)); // delay
        to.balance += amount;
    }
}
int main() { Account account1, account2; // two accounts for mutual transfers
            // concurrent transfers between accounts
            std::thread thr1(transferMoney, 50, std::ref(account1), std::ref(account2));
            std::thread thr2(transferMoney, 130, std::ref(account2), std::ref(account1));
            thr1.join(); thr2.join();
            std::cout << "account1.balance: " << account1.balance << std::endl;
            std::cout << "account2.balance: " << account2.balance << std::endl;
        }
    // account1.balance: 50 // 100 - 50. thr2 fails to execute for low funds
    // account2.balance: 150 // 100 + 50. thr2 fails to execute for low funds
```

- Correct result may be produced on occasions



Module Summary

Module M59

Partha Pratim Das

Objectives & Outlines

Threads

Races

Solution by Mutex

Solution by Lock

Solution by Atomic

Solution by Future

Solution by Async

Synchronization

Thread Local

Self-Study

Mutual Exclusion

Locks

Deadlock

Atomics

Condition Variables

Futures and Promises

Async

Practice Examples

Module Summary

- Understood synchronization issues in multi-thread programming in C++
- Studied various synchronization mechanisms through example
- Provided detail for self-study of synchronization mechanisms:
 - *Mutex*
 - *Lock*
 - *Atomics*
 - *Condition Variable*
 - *Future and Promises*
 - *Async*
- Explored use of the synchronization mechanisms to alleviate race condition and data race and left practice examples



Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Programming in Modern C++

Module M60: Closing Comments

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- Understood synchronization issues in multi-thread programming in C++
- Studied various synchronization mechanisms through example
- Provided detail for self-study of synchronization mechanisms:
 - *Mutex*
 - *Lock*
 - *Atomics*
 - *Condition Variable*
 - *Future and Promises*
 - *Async*
- Explored use of the synchronization mechanisms to alleviate race condition and data race and left practice examples



Module Objectives

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- Review C++ Course
- Key take-backs
- What next?



Module Outline

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

1 Objectives & Outlines

2 Course Summary

3 Modern C++ Features

- C++11 Features
- C++14 Features
- Deprecated Features

4 Key Take-back

- Prepare for Examination

5 Road Forward

6 Module Summary



Course Summary

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Course Summary



What we learnt in 59 Modules

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- **Programming in C++ is Fun**: C programs in C++, Equivalent programs [Module 01-05]
- **C++ as Better C**: Procedural Extensions of C [Module 06-10]
- **OOP in C++**: Object-Oriented Programming with Classes [Module 11-20]
- **Inheritance**: Generalization / Specialization of Object Modeling in C++ [Module 21-25]
- **Polymorphism**: Static & Dynamic Binding and Multiple Inheritance [Module 26-30]
- **Type Casting**: Cast Operators [Module 31-35]
- **Exceptions**: Error Handling in C & C++ [Module 36-37]
- **Templates**: Generic Programming in C++ [Module 38-40]
- **Streams**: File Handling in C & Stream in C++ for IO [Module 41-42]
- **STL**: Generic Programming Library in C++ Containers [Module 43-45]
- **Modern C++¹**: *General Features*: `auto`, `decltype`, initializer list, `constexpr`, rvalue & move semantics, `λ` & closure Objects, etc. [Module 46-53]
- **Modern C++**: *Class Features*: `=default`, `=delete`, delegating & inherited constructors, in-class initialization, etc. [Module 54-54]
- **Modern C++**: *Non-class Types & Template Features* [Module 55-55]
- **Modern C++**: *Resource Mgmt.*: `unique_ptr`, `shared_ptr`, `weak_ptr` [Module 56-57]
- **Modern C++**: *Concurrency Support*: Threads and synchronization [Module 58-59]

¹Mostly C++11, references to C++14 at time, occasionally to C++17 & C++20



What we learnt in 12 Tutorials

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- **How to build a C/C++ program?:** C Preprocessor (CPP), Build Pipeline, make Utility, Static and Dynamic Library [Tutorial 01-04]
- **Mixing C and C++ Code:** Issues and Resolutions, Project Example [Tutorial 05-06]
- **How to design a UDT like built-in types?:** Fraction UDT, Int & Poly UDT, Updates and Mixes of UDTs [Tutorial 07-09]
- **How to optimize C++11 programs using Rvalue and Move Semantics?** [Tutorial 10-10]
- **Compatibility of C and C++:** Significant Features, Summary [Tutorial 11-12]

Note that in some module and tutorial presentations some slides have been added after recording. These deal with more clarifications, examples, and minor added features. These are marked with "Post-Recording" in the slide title. So please refer to the presentation file along with the video



Upcoming Tutorials

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- **Embedded C++**
- **CMAKE**
- **Resource Management:** Managing memory and other resources by smart pointers, RAII & RDDI
- **C++ Coding Styles: How to write good code?**
- **C++ Processes and Tools:** Design, Development, Build (**Tutorial 01-04**), Debugging, Test, and Source Code & Version Management, and Bug Tracking
- **Design Patterns:** Singleton, Iterator, Command, Factory, Abstract Factory, Visitor, etc.
- **We may record more tutorials based on request from students. Feel free to suggest**



Modern C++ Features

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

Sources:

- [C++11](#), [isocpp.org](#)
- [C++14](#), [isocpp.org](#)
- [C++17](#), [isocpp.org](#)
- [C++20](#), [isocpp.org](#)
- [Modern cpp Features: C++20/17/14/11](#), [github](#)
- [Modern C++ Tutorial: C++11/14/17/20 On the Fly](#), Changkun Ou, O'Reilly, 2022

Modern C++ Features



C++ Standards

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

C++98	C++11	C++14	C++17	C++20
1998	2011	2014	2017	2020
Templates	Move Semantics	Reader-Writer Locks	Fold Expressions	Coroutines
STL with Containers and Algorithms	Unified Initialization	Generic Lambda Functions	constexpr if	Modules
Strings	auto and decltype		Structured Binding	Concepts
I/O Streams	Lambda Functions		std::string_view	Ranges Library
	constexpr		Parallel Algorithms of the STL	
	Multi-threading and Memory Model		File System Library	
	Regular Expressions		std::any, std::optional, and std::variant	
	Smart Pointers			
	Hash Tables			
	std::array			
ISO/IEC 14882:1998	ISO/IEC 14882:2011	ISO/IEC 14882:2014	ISO/IEC 14882:2017	ISO/IEC 14882:2020

Fixes on C++98: C++03: ISO/IEC 14882:2003, 2003
 Latest Version as of Sep-21: C++20: ISO/IEC 14882:2020, 2020

Partha Pratim Das



Modern C++ Features: C++11

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Modern C++ Features: C++11



Major C++11 Features: Core Language Features/1

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- `auto` and `decltype`
- trailing (suffix) return type
- list initialization (`initializer_list`), `std::initializer_list`
- uniform initialization: brace-or-equal initializers
- `enum class`: scoped enums
- `constexpr` and literal types
- `noexcept` specifier and operator
- `nullptr`
- `defaulted` and `deleted` functions
- delegating and inherited constructors
- `explicit` conversion
- `range-for` (based on Boost)
- `static_assert` (based on Boost)
- unicode string literals
- raw string literals
- user-defined literals
- `inline namespace`

[Module 46]

[Module 46]

[Module 47]

[Module 47]

[Module 55]

[Module 48]

[Module 48]

[Module 48]

[Module 54]

[Module 54]

[Module 54]

[Module 47]

[Module 48]

[Module 48]

[Module 48]

[Module 48]

[Module 48]



Major C++11 Features: Core Language Features/2

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- **rvalue reference and move semantics** [Module 49-51]
 - *move constructor, assignment operator*, `std::move`
 - *Perfect forwarding*, `std::forward`
- **lambda expressions** [Module 52-53]
- **concurrency support** [Module 58-59]
 - *threads*, `std::thread`
 - *synchronization*, `std::mutex`, `std::lock`, `std::atomic`, `std::condition_variable`, `std::future`, `std::promise`, `std::async`
 - *thread-local storage*, `thread_local`
- GC interface (removed in C++23)
- `long long`, `char16_t` and `char32_t` [Module 55]
- `final` and `override` [Module 54]
- type aliases [Module 55]
- variadic templates [Module 55]
- generalized (non-trivial) unions [Module 55]
- generalized PODs (trivial types and standard-layout types) [Module 55]
- attributes [Module 48]
- `alignof` and `alignas` [Module 48]



Major C++11 Features: Library Headers

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- `<array>`
- `<atomic>` [Module 58, 59]
- `<cfenv>`
- `<chrono>` [Module 58, 59]
- `<cinttypes>`
- `<condition_variable>` [Module 59]
- `<cstdint>` [Module 55]
- `<cuchar>`
- `<forward_list>`
- `<functional>` [Module 52, 53, 58]
- `<future>` [Module 58, 59]
- `<initializer_list>` [Module 47]
- `<mutex>` [Module 58, 59]
- `<random>` [Module 58, 59]
- `<ratio>`
- `<regex>`
- `<scoped_allocator>`
- `<system_error>`
- `<thread>` [Module 58, 59]
- `<tuple>`
- `<typeindex>`
- `<type_traits>`
- `<unordered_map>`
- `<unordered_set>`



Major C++11 Features: Library Features

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- atomic operations library
- `emplace()` and other use of rvalue references throughout all parts of the existing library
- Smart Pointers [Module 56]
 - `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`, `std::make_shared` [Module 57]
- `std::move_iterator`
- `std::initializer_list` [Module 47]
- stateful and scoped allocators
- `std::forward_list`
- chrono & ratio library [Module 58, 59]
- Unicode conversion facets
- thread library [Module 58-59]
 - `std::thread`, `std::mutex`, `std::lock`, `std::atomic`, `std::condition_variable`, `std::future`, `std::promise`, `std::async`, `thread_local`
- `std::function` [Module 52, 53, 58]
- `std::bind` [Module 58]
- `std::exception_ptr`
- `std::error_code` and `std::error_condition`
- iterator improvements: `std::begin`, `std::end`, `std::next`, `std::prev` [Module 03, 05, 43-45]
- Unicode conversion functions



Major C++11 Features: Library Features

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- New Algorithms:

- `std::all_of`, `std::any_of`, `std::none_of`,
- `std::find_if_not`, `std::copy_if`, `std::copy_n`,
- `std::move`, `std::move_backward`,
- `std::random_shuffle`, `std::shuffle`,
- `std::is_partitioned`, `std::partition_copy`, `std::partition_point`,
- `std::is_sorted`, `std::is_sorted_until`, `std::is_heap`, `std::is_heap_until`,
- `std::minmax`, `std::minmax_element`,
- `std::is_permutation`,
- `std::iota`,
- `std::uninitialized_copy_n`



Modern C++ Features: C++14

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Modern C++ Features: C++14



Major C++14 Features: Language

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- Binary literals
- Generalized return type deduction
- `decltype(auto)`
- Generalized lambda captures
- Generic lambdas
- Variable templates
- Extended `constexpr`
- The [`deprecated`] attribute
- Digit separators

[Module 48]

[Module 51]

[Module 46]

[Module 53]

[Module 53]

[Module 55]

[Module 48]

[Module 48]

[Module 48]



Major C++14 Features: Library

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- Shared locking
- User-defined literals for `std::` types
- `make_unique`
- Type transformation `_t` aliases

[Module 48]

[Module 57]



Major Deprecated Features of C++98 / C++03

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- String literal constant is no longer allowed to be assigned to a `char*`. If you need to assign and initialize a `char*` with a string literal constant, use `const char*` or `auto`
- C++98 exception description, `unexpected_handler`, `set_unexpected()` and other related features are deprecated and should use `noexcept` [Module 48]
- `auto_ptr` is deprecated and `unique_ptr` should be used [Module 57]
- `register` keyword is deprecated and if used there is no practical meaning
- `operator++` for `bool` type is deprecated
- If a class has a destructor, the properties for which it generates copy constructors and copy assignment operators are deprecated [Module 51]
- C language style type conversion using `(convert_type)` before variables is deprecated, and `static_cast`, `reinterpret_cast`, `const_cast` should be used for type conversion
- Some of the C standard libraries that can be used are deprecated in the C++17, such as `<ccomplex>`, `<cstdalign>`, `<cstdbool>` and `<ctgmath>`
- Parameter binding, `export` are deprecated for `std::bind`, `std::function` [Module 58, 53]
- ... and many more

Note: *Deprecation is not completely unusable, it implies that features will disappear from future standards and should be avoided. But, the deprecated features are still part of the standard library, and most of the features are actually **permanently** reserved for compatibility reasons*



Key Take-back

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Key Take-back



What have we learnt?

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- C++ is multi-paradigm
 - Procedural: Better C
 - Object-Oriented: Encapsulation, Inheritance, and Polymorphism
 - Generic: Templates
- Reuse is Key
 - Library functions
 - Function Overloading (Static Polymorphism)
 - Inheritance & Dynamic Polymorphism
 - Templates
 - STL Containers and Algorithms
- Designing good data types is a key for good programming in C++
- While programming in C++, we should keep an eye on:
 - Efficiency
 - Safety
 - Clarity
- Move with the standards: C++03 → C++11 → C++14 → C++17 → C++20 → C++23

Do not write C-style programs using C++ compiler



Important References

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- [C++ reference](#), cppreference
- [CPlusPlus](#), cplusplus
- [An Overview of the New C++ \(C++11/14\)](#), Training Courses; [Effective Modern C++](#), 2015; [Effective C++](#), 3rd Ed., 2005 and [More Effective C++](#), 1st Ed., 1996; [Effective STL](#), 1st Ed., 2001, Scott Meyers
- [Bjarne Stroustrup's FAQ](#); [C++11 - the new ISO C++ standard](#), stroustrup
- [Andrei Alexandrescu: Creator of D](#); [Modern C++ Design](#), Andrei Alexandrescu, 2001; [C++ Coding Standards](#), 1st Ed., Herb Sutter and Andrei Alexandrescu, 2004; [The D Programming Language](#), Andrei Alexandrescu, 2010
- [Herb Sutter: Sutter's Mill](#): Chair of ISO C++ standards committee for over a decade; [Exceptional C++](#), 1999; [More Exceptional C++](#), 2001 by Herb Sutter
- [C++ Templates](#), 2nd Ed., D. Vandevoorde, N. M. Josuttis, and D. Gregor, 2017
- [The C++ Standard Library: A Tutorial and Reference](#), 2nd Ed., Nicolai M. Josuttis, 2012
- [Google C++ Style Guide](#)
- [Modern C++ Tutorial: C++11/14/17/20 On the Fly](#), Changkun Ou, O'Reilly, 2022



Key Take-back: Prepare for Examination

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Key Take-back: Prepare for Examination



Prepare for Examination

Module M60

Partha Pratim Das

Objectives & Outlines

Course Summary

Modern C++ Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for Examination

Road Forward

Module Summary

- Watch the Videos
- Revise the Assignments and Solutions
- Practice lots and lots of coding with every feature
- Design and implement complete data types – Complex, Fraction, Vector, Matrix, Polynomial etc. using copy / move semantics
- Use λ 's and try simple multi-threaded programming



Road Forward

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

Road Forward



Road Forward

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- Learn the topics not covered
- Breathe programming – regularly code and implement systems
- Read lots and lots of programs by good coders
- Learn Python / Java
- Study **Object Oriented Analysis and Design**
- Study **Unified Modeling Language**
- Study **Software Engineering**
- Study **Books and References** mentioned



Module Summary

Module M60

Partha Pratim
Das

Objectives &
Outlines

Course Summary

Modern C++
Features

C++11 Features

C++14 Features

Deprecated Features

Key Take-back

Prepare for
Examination

Road Forward

Module Summary

- Course on Modern C++ concluded



Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

Programming in Modern C++

Tutorial T12: Compatibility of C and C++: Part 2: Summary

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Recap

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

- We have understood why C and C++ incompatible across dialects in spite of C++ being an intended super-set of C
- We studied specific incompatibilities over nearly two dozen features
- We discussed some workarounds to write more compatible code between C and C++



Tutorial Objectives

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

- We present a summary of differences between C and C++



Tutorial Outline

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

1 Tutorial Recap

2 Summary of Compatibility

3 Tutorial Summary



Summary of Compatibility

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

Summary of Compatibility

We summarize the incompatibility in features already discussed, and also introduce a few new ones in brief



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

C Feature	C++ Feature
● Implicit Conversion of <code>void*</code> is allowed in C	● Implicit conversion is not allowed in C++; allowed only with explicit cast
● Implicit Discard of <code>const</code> qualifier for pointer is allowed in C	● Implicit discard is not allowed in C++; allowed only with explicit cast
● Initialization of <code>const</code> Variable is optional in C	● Initialization is mandatory in C++
● C Standard Library functions have unique signature. For example, <code>strchr</code> in <code>string.h</code>	● In C++, they may have additional overloaded functions. For example, <code>strchr</code> in <code>cstring</code>
● Implicit Conversion of <code>int</code> to <code>enum</code> is allowed in C	● Implicit conversion is not allowed in C++; allowed only with explicit cast
● <code>enum</code> Enumerators are of type <code>int</code> in C	● Enumerators are of distinct types in C++, having different size from <code>int</code>
● Multiple definitions of a global in a single translation unit is allowed in C	● It is disallowed in C++ due to One Definition Rule (ODR)
● Declaring a new type having same name as an existing <code>struct</code>, <code>union</code> or <code>enum</code> is allowed in C	● It is disallowed in C++ as all declarations of such types carry the <code>typedef</code> implicitly
● In C, a function prototype without parameters implies that the parameters are unspecified, and can be called with zero or more parameters	● In C++, it means zero parameter only

For compatibility, use **`void`** for parameter when there is no parameter



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Summary of Compatibility

Tutorial Summary

C Feature	C++ Feature
● Character literals like 'a' are of type <code>int</code> in C. Hence: 1. <code>sizeof('a') = sizeof(int)</code> 2. 'a' is always a signed expression, regardless of whether or not <code>char</code> is a signed or unsigned type	● They are of type <code>char</code> in C++. Hence: 1. <code>sizeof('a') = sizeof(char) = 1</code> 2. If 'a' a signed expression or not depends on whether <code>char</code> is a signed or unsigned type, which is implementation specific
● Boolean type <code>bool</code> is supported in <code>C99</code> with constants <code>true</code> and <code>false</code>. In <code>C99</code>, a new keyword, <code>_Bool</code>, is introduced as the new Boolean type. The header <code>stdbool.h</code> provides macros <code>bool</code>, <code>true</code> and <code>false</code> that are defined as <code>_Bool</code>, <code>1</code> and <code>0</code>, respectively. Therefore, <code>true</code> and <code>false</code> have type <code>int</code> in C	● In C++, <code>bool</code> is a built-in type with constants <code>true</code> and <code>false</code>. All these are reserved keywords. Conversions to <code>bool</code> are similar to C
● For Nested structs, the inner <code>struct</code> is also defined outside the outer <code>struct</code> in C	A nested <code>struct</code> is defined only within the scope / namespace of the outer <code>struct</code> in C++



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Summary of Compatibility

Tutorial Summary

C Feature

- **inline Function** is supported in **C99**. It is a directive that suggests (but does not require) that the compiler substitute the body of the function inline by inline expansion (saving the overhead of a function call). But it complicates the linkage behavior:

```
#include <stdio.h>
inline int foo() { return 2; } /* Inline in C */
int main() { int ret; /* Driver code */
    ret = foo();          /* inline function call */
    printf("Output is: %d", ret);
}
```

It gives a linker error **undefined reference to 'foo'** - as GCC **inlines**, there is no function call present (**foo**) inside **main**. Hence, we fix as:

```
#include <stdio.h>
static /* Inline bound to the this file - no extern linkage */
inline int foo() { return 2; } /* Inline in C */
int main() { int ret; /* Driver code */
    ret = foo();          /* inline function call */
    printf("Output is: %d", ret);
}
```

C++ Feature

- In C++, the external linkage issues of **inline** functions are handled by the compiler:

```
#include <cstdio>
using namespace std;
inline int foo()
{ return 2; } // Inline in C++
int main() { // Driver code
    int ret;
    ret = foo(); // inline call
    printf("Output is: %d", ret);
}
```

Source: [inline function specifier](#) Accessed 14-Sep-21



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

C Feature	C++ Feature
• Variable Length Array (VLA) is supported from C99	• Supported if array size is a constant-expression in C++11 standard and simple expression (not constant-expression) in C++14 standard
• Flexible Array Member (FAM) is supported from C99	• Not supported in ISO C++
• restrict type qualifier for pointer declarations is supported from C99	• Not supported in ISO C++, but compilers like GCC, Visual C++, and Intel C++ provide similar functionality as an extension
• Complex arithmetic using the float complex and double complex primitive data types was added in the C99 standard, via the _Complex keyword and complex convenience macro	• In C++, complex arithmetic can be performed using the complex number class, <i>but the two methods are not code-compatible</i>. (The standards since C++11 require binary compatibility)
• Array parameter qualifiers in functions is supported from C89: <pre>int foo(int a[const]); // equivalent to int *const a int bar(char s[static 5]); // s is at least 5 chars long</pre>	• Not supported in ISO C++



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Summary of Compatibility

Tutorial Summary

C Feature	C++ Feature
From C89, Compound Literals is generalized to both built-in and user-defined types by the list initialization syntax of C++11, <i>although with some syntactic and semantic differences</i>: <pre>struct X { int p, q; }; /* Equivalent in C++ would be X{4, 6} */ struct X a = (struct X){4, 6};</pre>	The following works in C++11 onward: <pre>struct X { int p, q; }; struct X a = (struct X){4, 6}; struct X b = X{4, 6};</pre>
From C89, Designated Initializers for structs and arrays are allowed in C. Designated initializers allow members to be initialized by name, in any order, and without explicitly providing the preceding values: <pre>struct s { int x; float y; char *z; }; struct s pi_by_order = { 3, 3.1415, "Pi" }; struct s pi_by_name = { .z = "Pi", .x = 3, .y = 3.1415 }; /* Only C */ char s[20] = {[0] = 'a', [8] = 'g'}; /* Only C */ #define MAX 10 int a[MAX] = { 1, 3, 5, 7, 9, [MAX-5] = 8, 6 };</pre>	These are not allowed in C++. struct designated initializers are planned for addition in C++2x <pre>struct s { int x; float y; char *z; }; struct s pi_by_order = { 3, 3.1415, "Pi" }; struct s pi_by_name = { .z = "Pi", .x = 3, .y = 3.1415 }; // C++2x ? char s[20] = {[0] = 'a', [8] = 'g'}; // No C++ Plan #define MAX 10 int a[MAX] = { 1, 3, 5, 7, 9, [MAX-5] = 8, 6 };</pre>



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Summary of Compatibility

Tutorial Summary

C Feature	C++ Feature
• <code>_Noreturn</code> function specifier marks function in C that does not return with a value or implicitly after completion. It may return by executing <code>longjmp</code>: <code>#include <stdio.h></code> <code>/* Nothing to return */ _Noreturn void show() { printf("BYE BYE"); } int main() { printf("Ready to begin..."); show(); printf("NOT over till now"); }</code> Compiler Warning: <code>'noreturn' function does return</code> Output is: <code>Ready to begin...BYE BYE</code> Source: _Noreturn function specifier Accessed 14-Sep-21	• <code>[[noreturn]]</code> attribute marks function in C++ that does not return with a value or implicitly after completion. It may throw: <code>#include <cstdio> using namespace std;</code> <code>/* Nothing to return */ [[noreturn]] void show() { printf("BYE BYE"); } int main() { printf("Ready to begin..."); show(); printf("NOT over till now"); }</code> Compiler Warning: <code>'noreturn' function does return</code> Output: <code>Ready to begin...BYE BYE</code> Source: C++ attribute: noreturn Accessed 14-Sep-21

This must not be confused with void return type used for functions that return, but without a value



Compatibility of C and C++: Summary

Tutorial T12

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Summary of Compatibility

Tutorial Summary

C Feature	C++ Feature
● Extra C++ reserved words may fail C codes in C++ compiler. The following code works fine in C. <pre>struct template { int new; struct template* class; };</pre>	● It naturally fails in C++: <pre>struct template { // template is a reserved word int new; // new is a reserved word struct template* class; // class is a reserved word };</pre>
● We can observe several mixed effects in C and C++ due to incompatibility where the code compiles in both languages but behaves differently <pre>#include <stdio.h> extern int T; int size(void) { struct T { int i; int j; }; return sizeof(T); /* C: return sizeof(int) */ } int main() { printf("%d", size()); }</pre>	● It naturally fails in C++: <pre>#include <stdio> using namespace std; extern int T; int size(void) { struct T { int i; int j; }; return sizeof(T); // C++: return sizeof(struct T) } int main() { printf("%d", size()); }</pre>



Tutorial Summary

Tutorial T12

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Summary of
Compatibility

Tutorial Summary

- We presented a summary of differences table between **C99** and **C++11**